Recurrent Neural Networks: The Memory Breakthrough

Danna Gurari
University of Colorado Boulder
Spring 2026

BY NC SA

https://dannagurari.colorado.edu/course/neural-networks-and-deep-learning-spring-2026/

Review

- Last lecture:
  - Vision: multi-layer networks provide sufficient representational power
  - Breakthrough: gradients make multi-layer networks trainable
  - Machine abstraction: neural networks as computational graphs
  - Programming tutorial: developing neural networks in PyTorch
- Assignments (Canvas):
  - Lab assignment 1 due Friday
- Questions?

Today's Topics

- Limitation of feedforward neural networks: supporting sequential data
- Key insight: *recurrent* neural networks (RNNs)
- Breakthrough: gated RNNs for dynamically deciding what to remember
- Programming tutorial: recurrent neural networks in PyTorch

Today's Topics

- Limitation of feedforward neural networks: supporting sequential data
- Key insight: *recurrent* neural networks (RNNs)
- Breakthrough: gated RNNs for dynamically deciding what to remember
- Programming tutorial: recurrent neural networks in PyTorch

Illustrative Example: Parity Problem

- Given a sequence, answer if the number of 1s is even or odd

- Let’s try it…

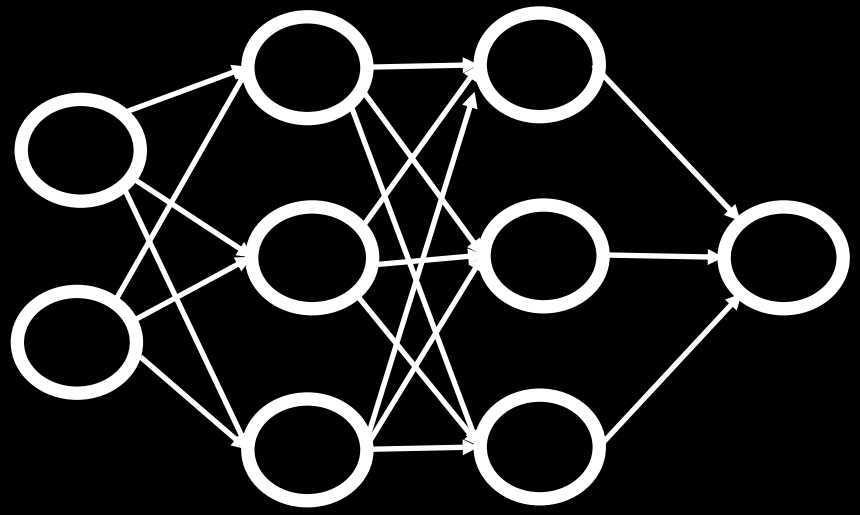


Illustrative Example: Parity Problem

- Given a sequence, answer if the number of 1s is even or odd
- Let's try it… 

- Necessary skill: memory to know if current count is even or odd when each new ball arrives

Illustrative Example: Parity Problem



Why are feedforward networks a poor fit for this sequential problem?

- **Lacks memory** (amnesia occurs after *every* forward pass)
- **Input size is fixed** (doesn't support arbitrary sequence lengths)

Critical Gap of Neural Networks: Support for Real-World Sequential Problems

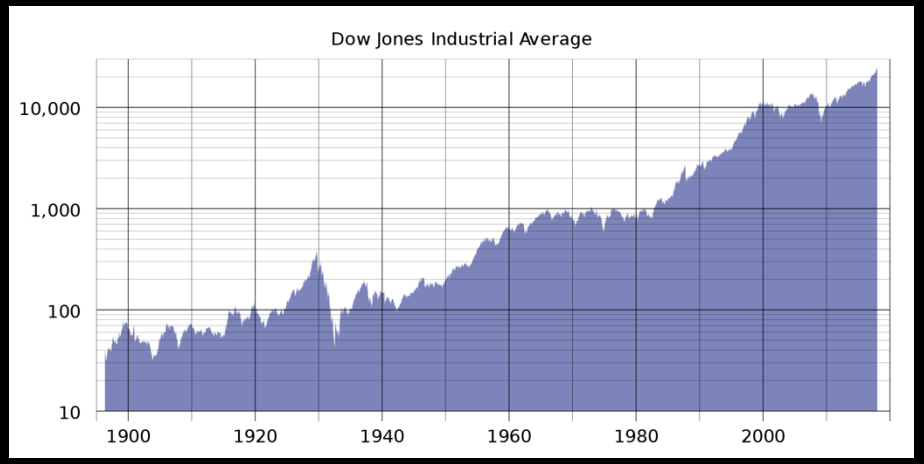
Weather Forecasting



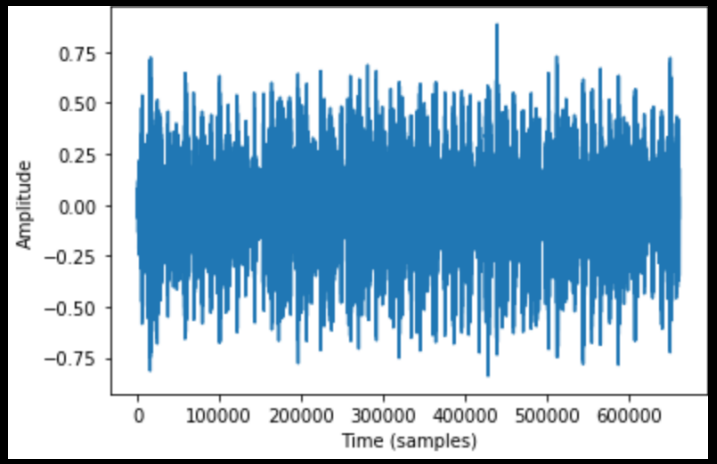
Text Analysis



Stock Market



Audio Processing



Critical Gap of Neural Networks: Three Types of Real-World Sequential Problems

One-to-Many
Input: fixed-size
Output: sequence
e.g., image captioning



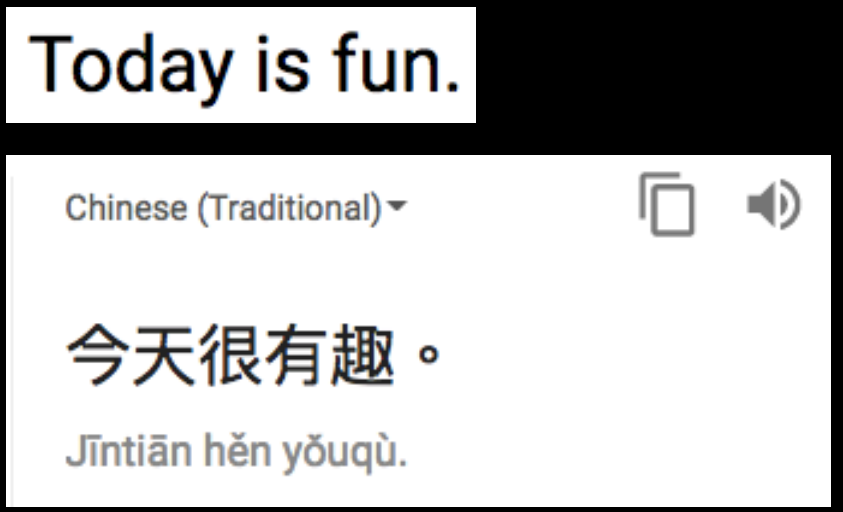
A digital thermometer resting on a wooden table, showing 38.5 degrees Celsius.

Many-to-One
Input: sequence
Output: fixed-size
e.g., sentiment analysis



Positive? Negative? Neutral?

Many-to-Many
Input: sequence
Output: sequence
e.g., language translation



Today's Topics

- Limitation of feedforward neural networks: supporting sequential data
- Key insight: *recurrent* neural networks (RNNs)
- Breakthrough: gated RNNs for dynamically deciding what to remember
- Programming tutorial: recurrent neural networks in PyTorch

Context: Emergence of “Recurrent” Connections

- Recurrent connections: an approach to save information for later use (i.e., memory)
- 1962: discussed in Rosenblatt’s book on perceptrons
- 1982: promising experimental results demonstrated by John Hopfield (2024 Nobel Prize winner)
- 1986: generalizable learning method (backpropagation) introduced in Rumelhart paper →

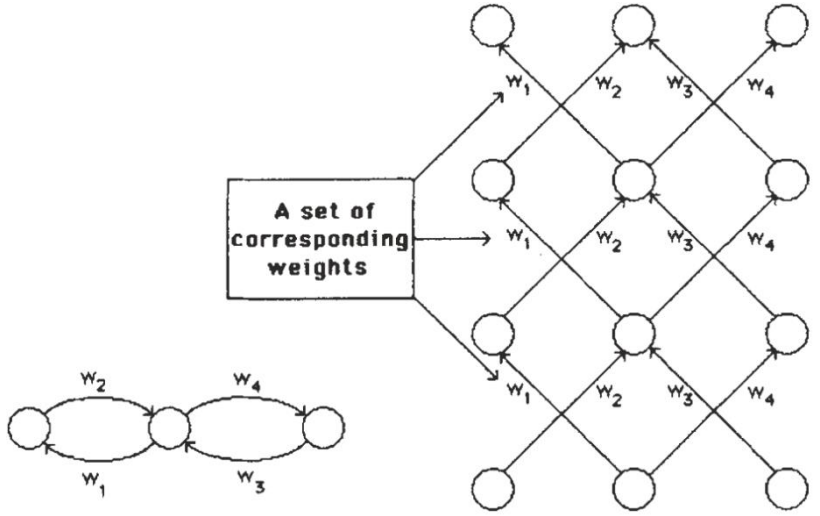
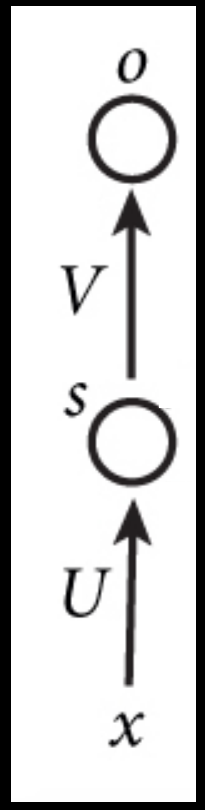


Fig. 5 A synchronous iterative net that is run for three iterations and the equivalent layered net. Each time-step in the recurrent net corresponds to a layer in the layered net. The learning procedure for layered nets can be mapped into a learning procedure for iterative nets. Two complications arise in performing this mapping: first, in a layered net the output levels of the units in the intermediate layers during the forward pass are required for performing the backward pass (see equations (5) and (6)). So in an iterative net it is necessary to store the history of output states of each unit. Second, for a layered net to be equivalent to an iterative net, corresponding weights between different layers must have the same value. To preserve this property, we average $\partial E/\partial w$ for all the weights in each set of corresponding weights and then change each weight in the set by an amount proportional to this average gradient. With these two provisos, the learning procedure can be applied directly to iterative nets. These nets can then either learn to perform iterative searches or learn sequential structures[4].

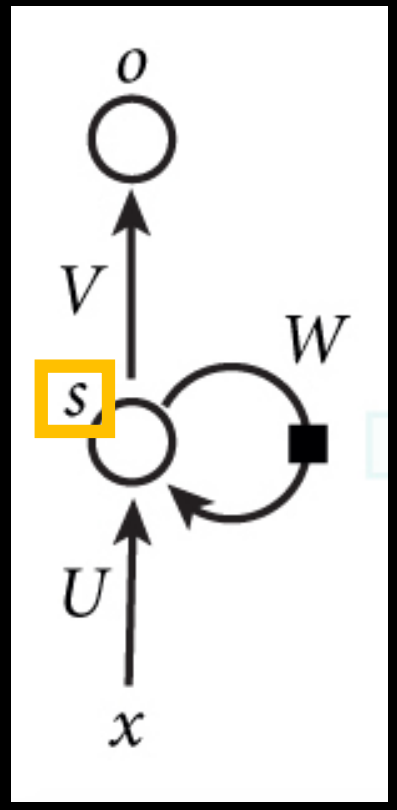
Recurrent Neural Networks (RNNs)

- Main idea: add hidden state for storing information about the past (i.e., memory)

Feedforward Network
Each layer receives input from the previous layer



Recurrent Network
Each layer receives input from the previous layer and output from the previous time step (i.e., time-delayed connections)



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

Recurrent Neural Networks (RNNs)

- Main idea: add hidden state for storing information about the past (i.e., memory)

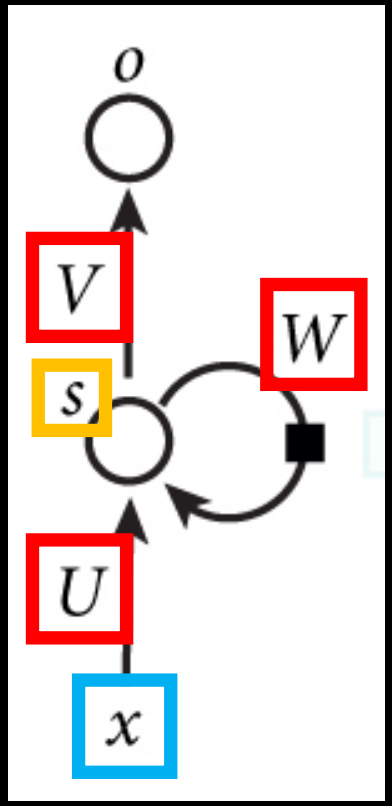
Recurrent: same function is applied at previous time

Model parameters

$s_t = f_m(s_{t-1}, x_t)$

New state | Old state | Input at time step

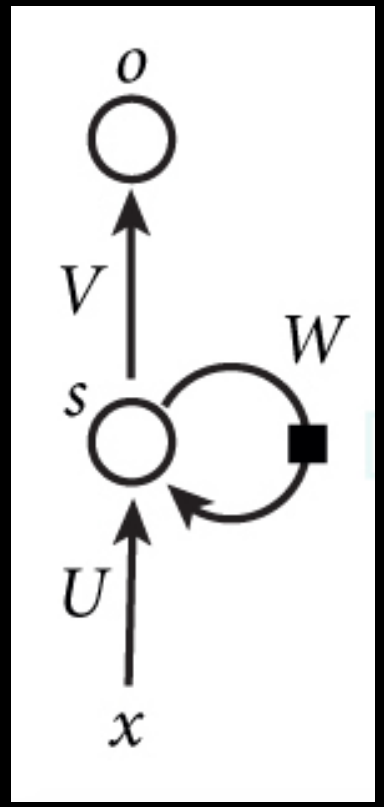
Recurrent Network
Each layer receives input from the previous layer and output from the previous time step (i.e., time-delayed connections)



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

RNN: Time Step 1

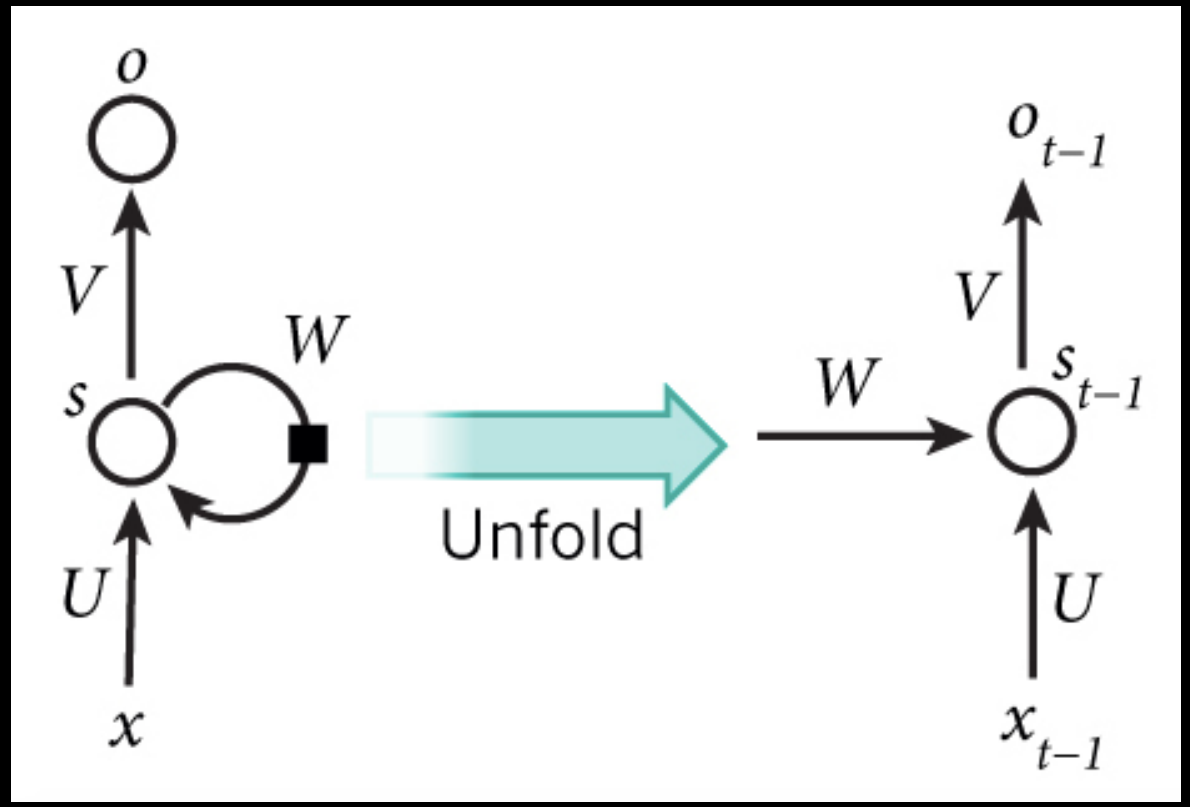
- Main idea: add hidden state for storing information about the past (i.e., memory)



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

RNN: Time Step 1

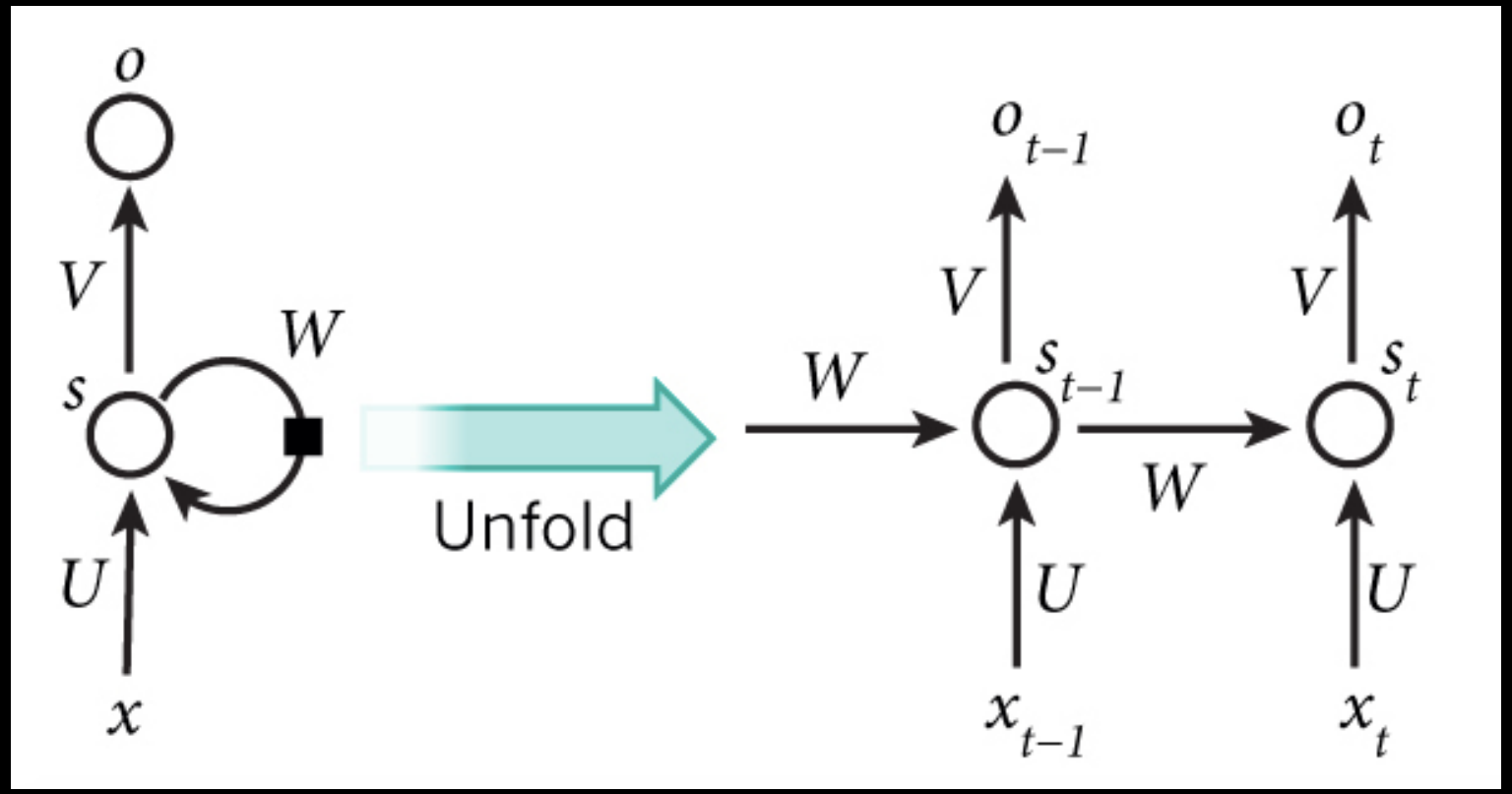
- Main idea: add hidden state for storing information about the past (i.e., memory)



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

RNN: Time Step 2

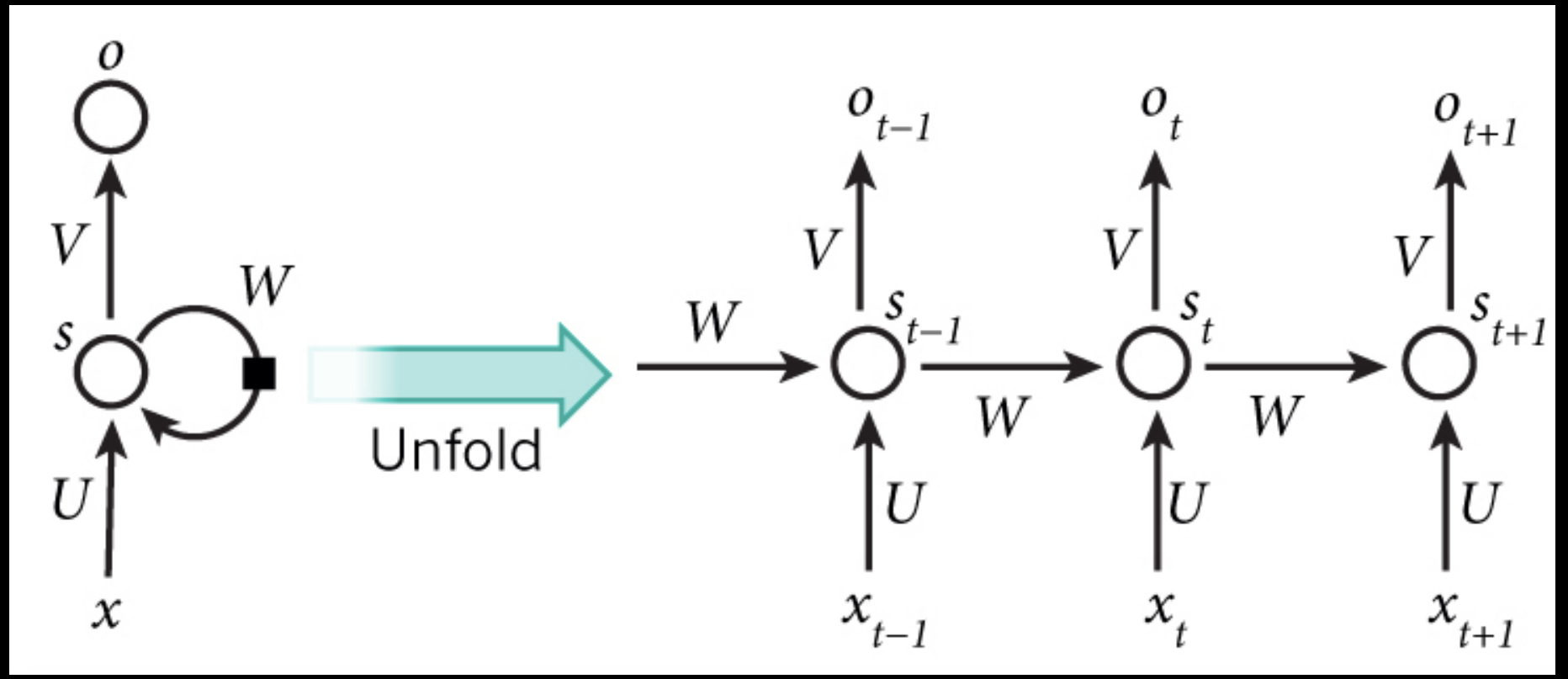
- Main idea: add hidden state for storing information about the past (i.e., memory)



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

RNN: Time Step 3

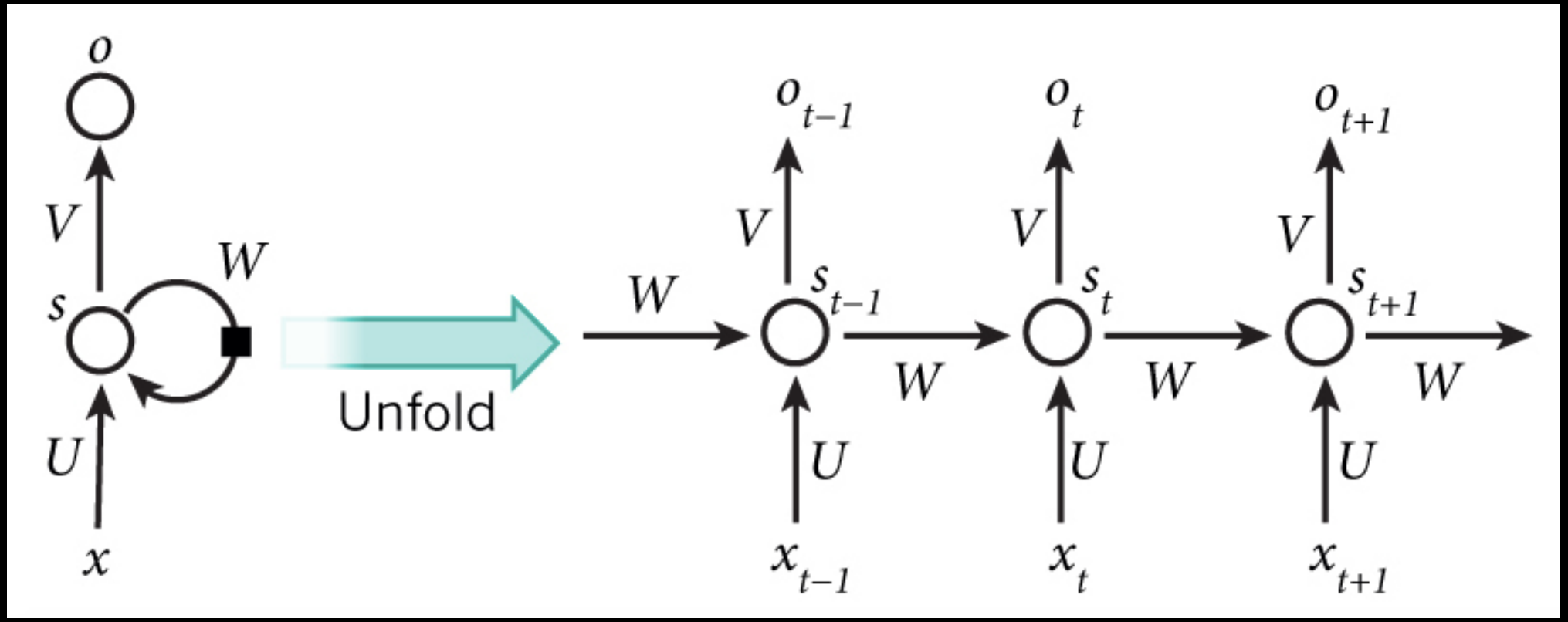
- Main idea: add hidden state for storing information about the past (i.e., memory)



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

RNN: And So On…

- Main idea: add hidden state for storing information about the past (i.e., memory)

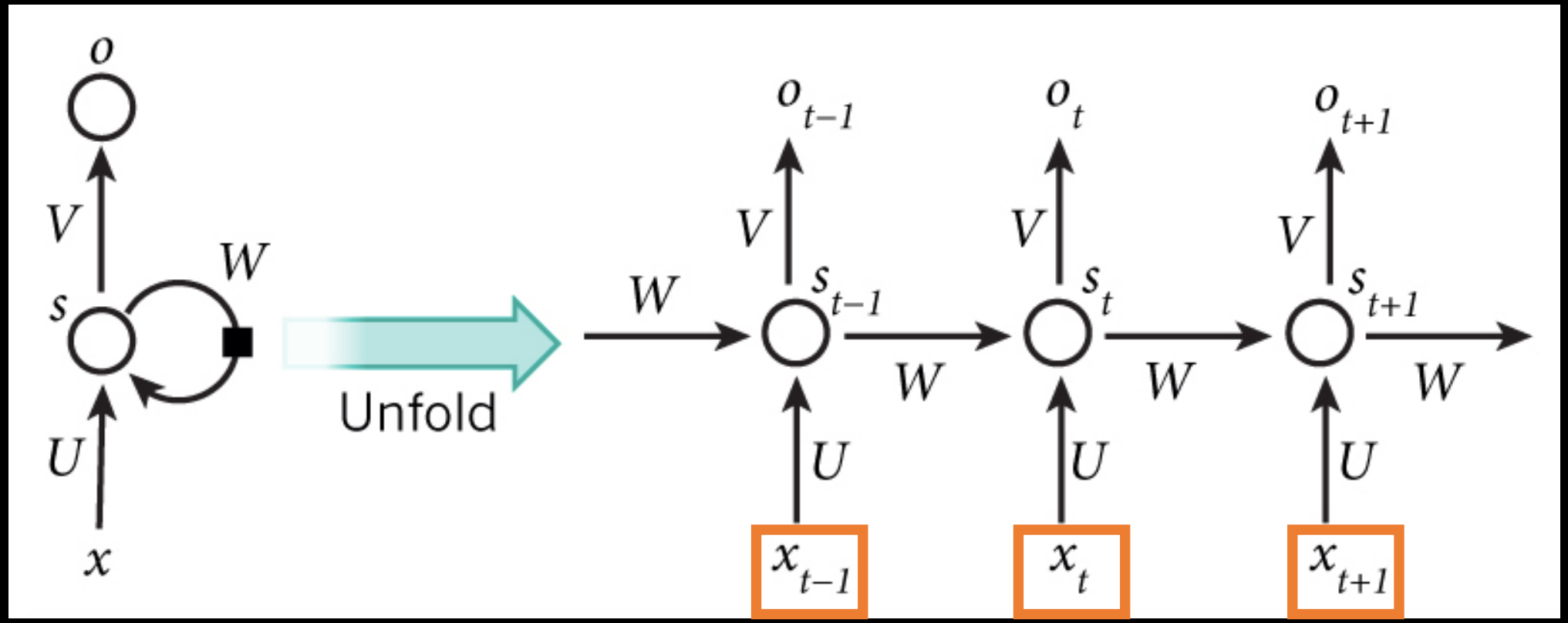


Unfolding enables the recursive equation to be represented as a directed acyclic computational graph

http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

RNN: Arbitrary Sequence Support

- Can theoretically handle any input size by unfolding more/less time steps



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

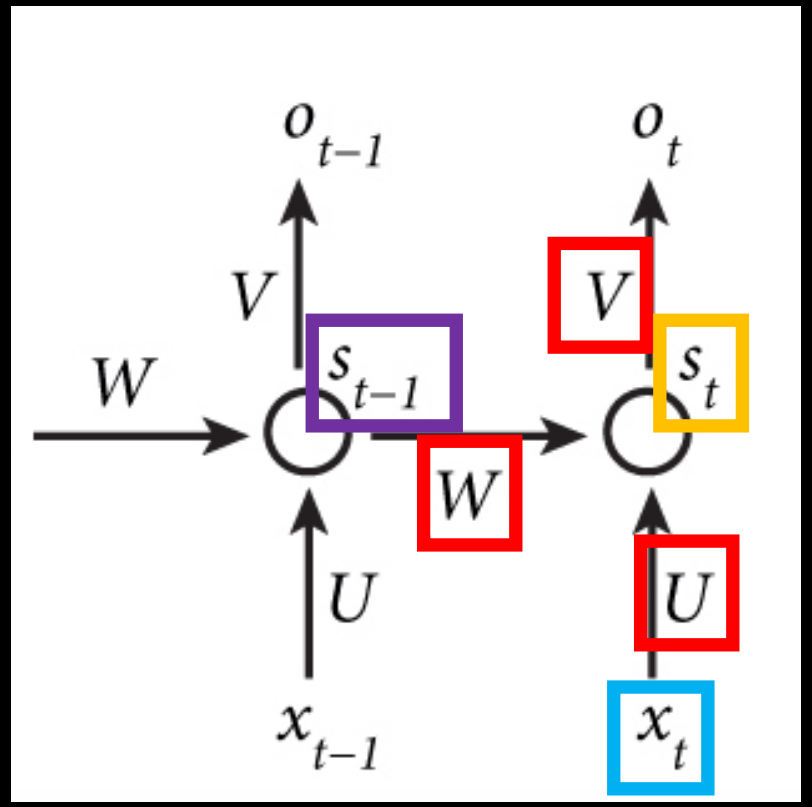
RNN: Model Parameters and Inputs

Recurrence formula applied at every time step:

Model parameters

$s_t = f_m(s_{t-1}, x_t)$

New state | Old state | Input at time step



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

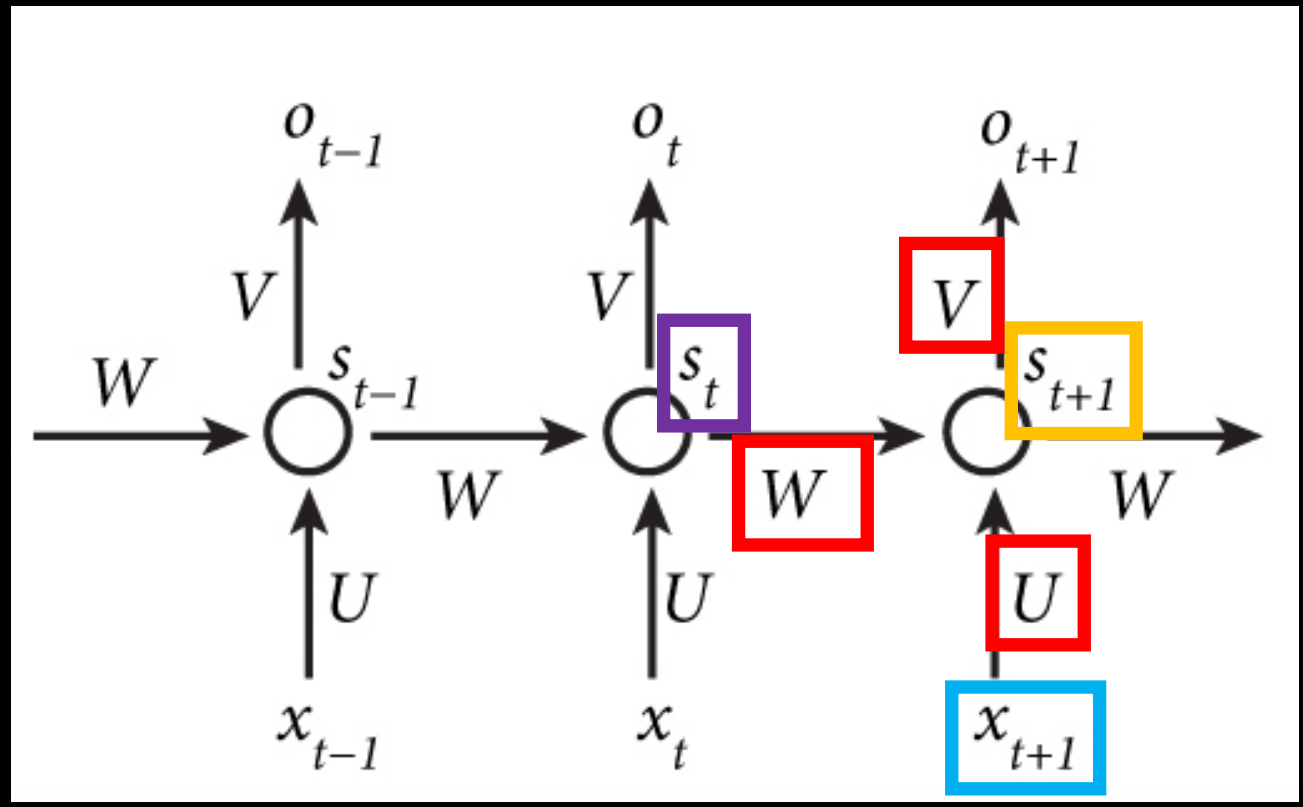
RNN: Model Parameters and Inputs

Recurrence formula applied at every time step:

Model parameters

$s_t = f_m(s_{t-1}, x_t)$

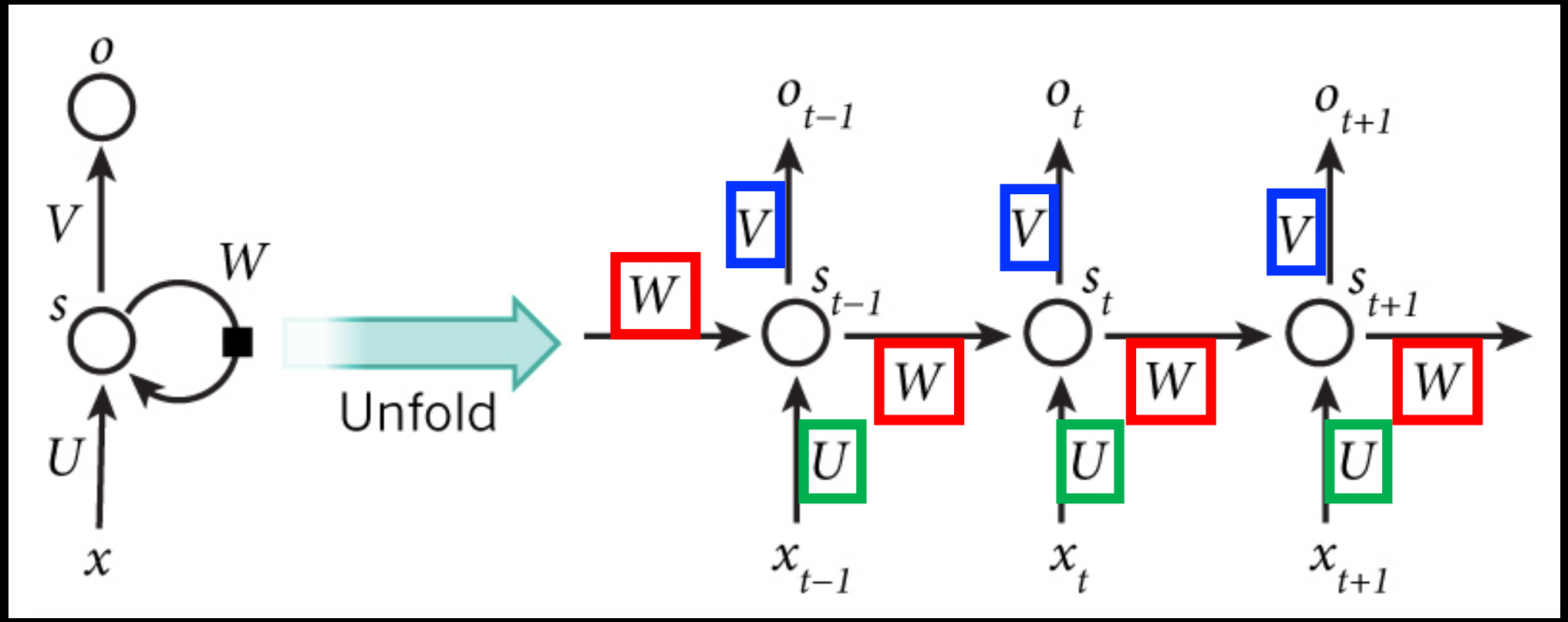
New state | Old state | Input at time step



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

RNN: Model Parameters and Inputs

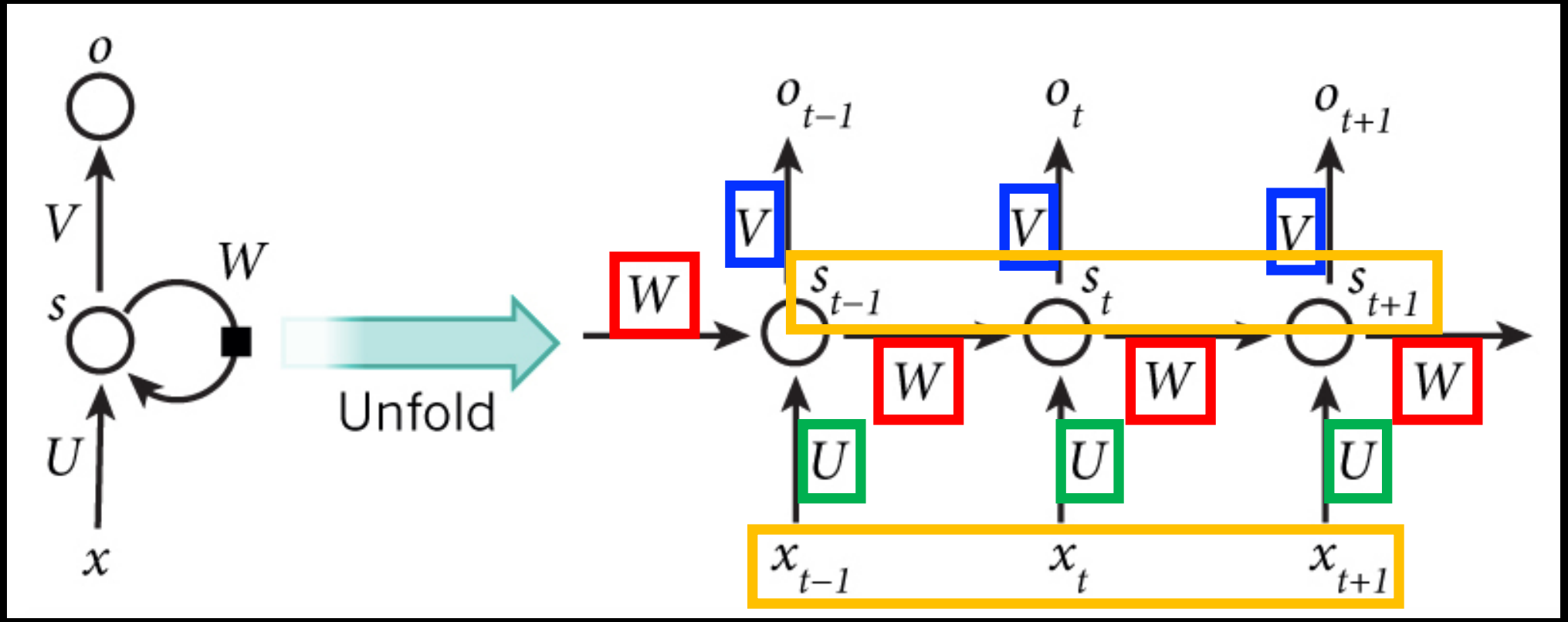
- When unfolded, a RNN is a deep feedforward network with shared weights!



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

RNN: Model Parameters and Inputs

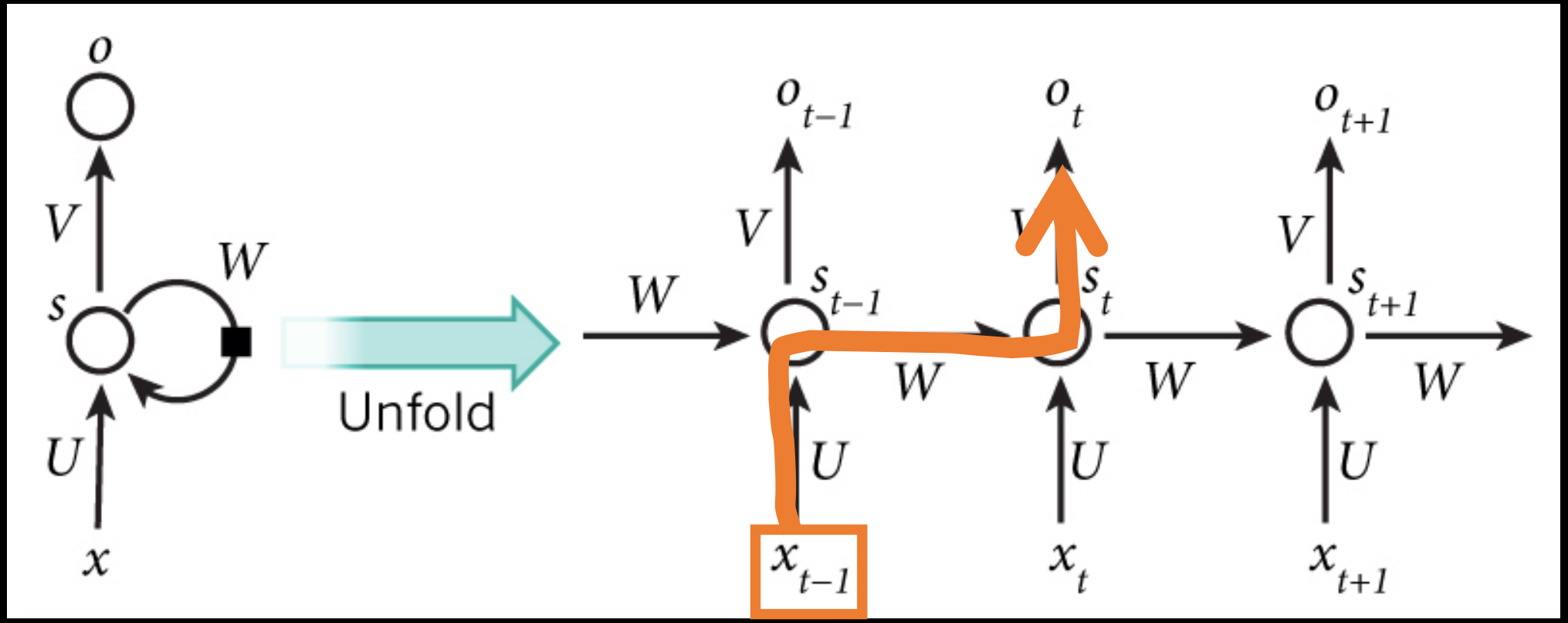
- If all layers share the same parameters (U, V, W), what differs between layers?



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

RNN: Memory Support

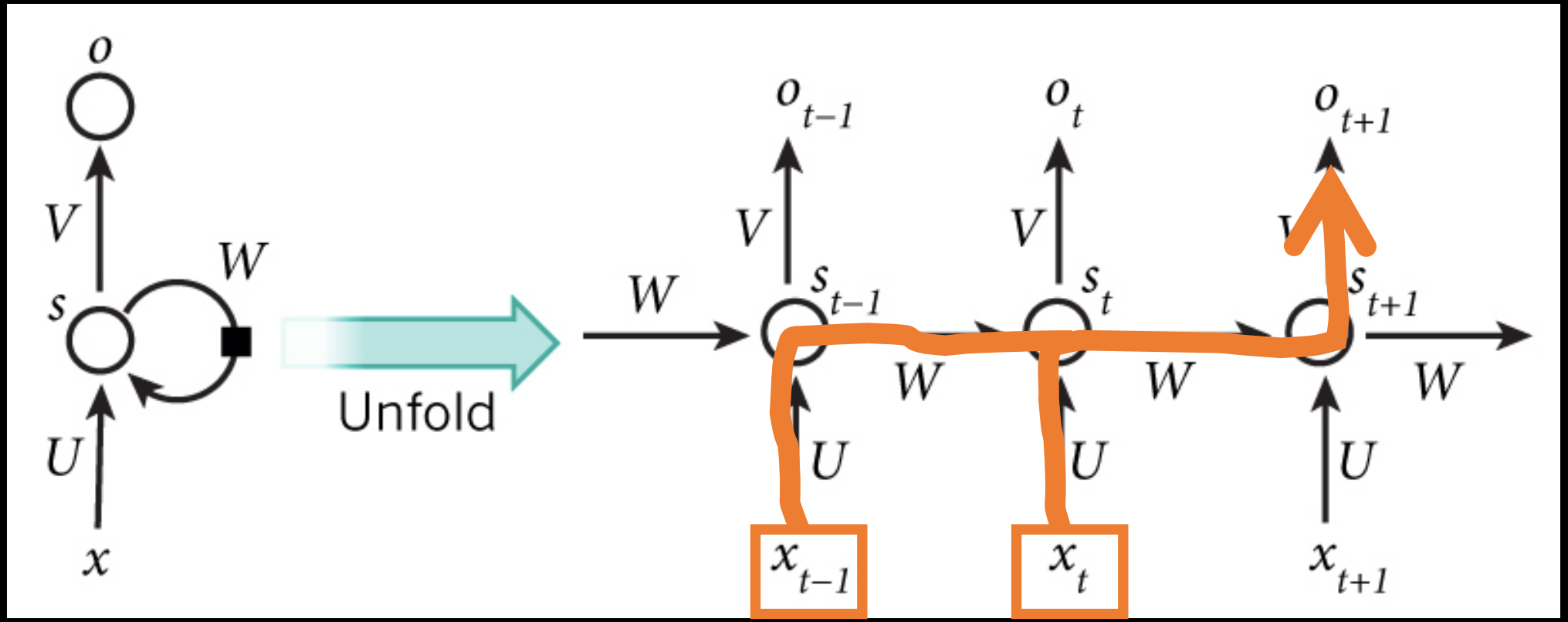
- Remembers past inputs for a duration determined by model's weights and input data



http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

RNN: Memory Support

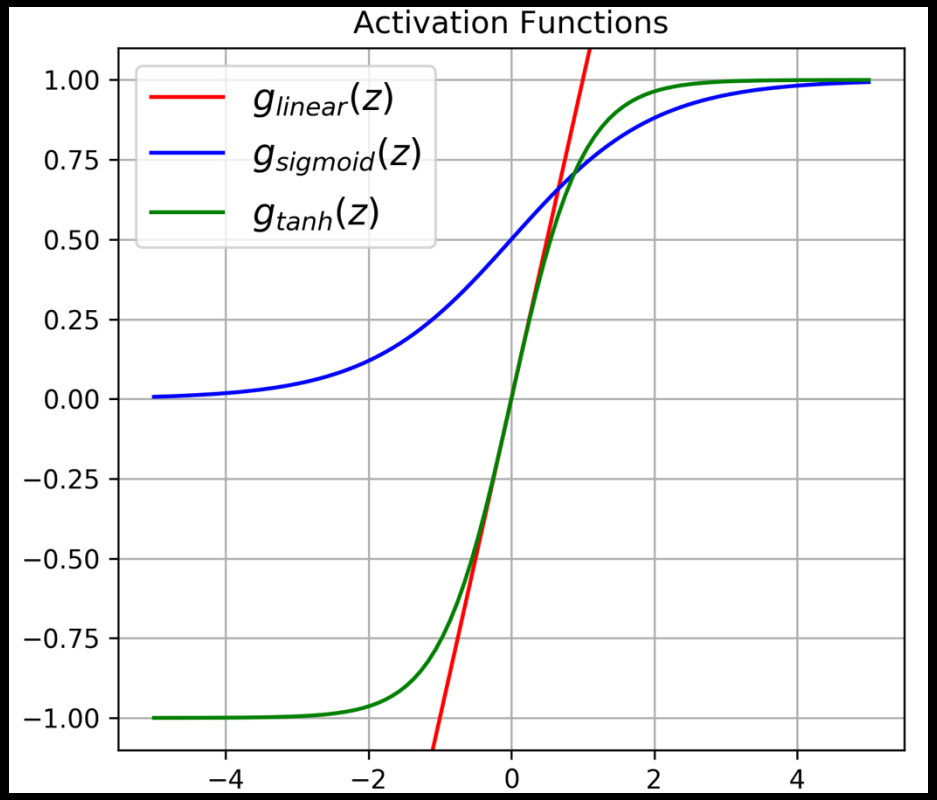
- Remembers past inputs for a duration determined by model's weights and input data

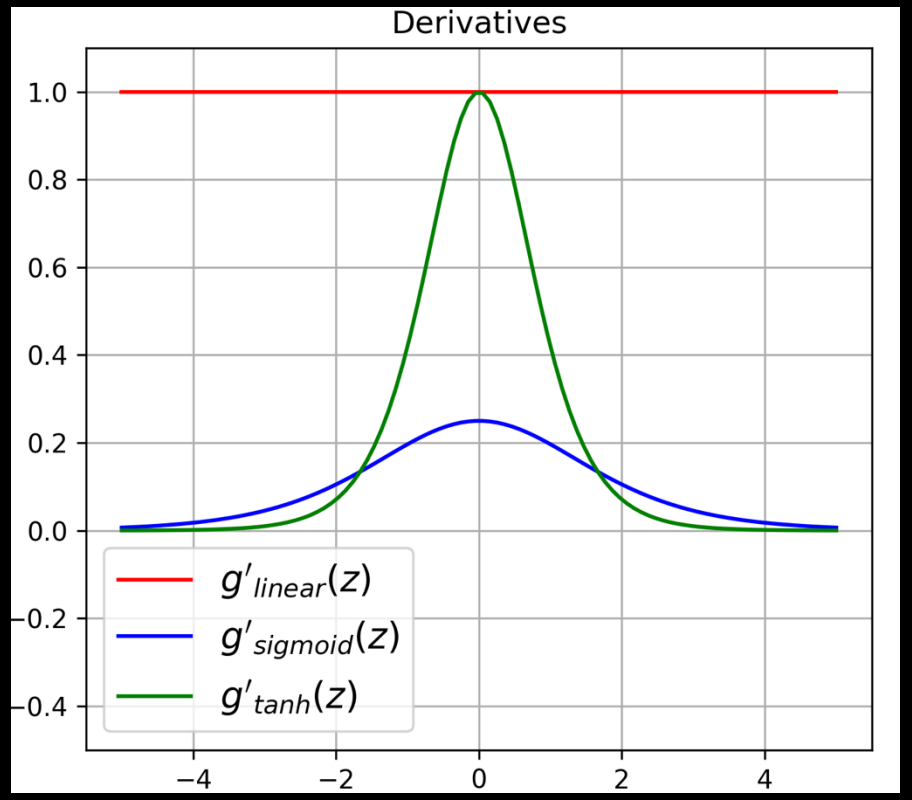


http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

Example 1: Predict Parity

- RNNs commonly use tanh activation function (for hidden neurons)
  - What range of activation values can a sigmoid vs tanh represent?

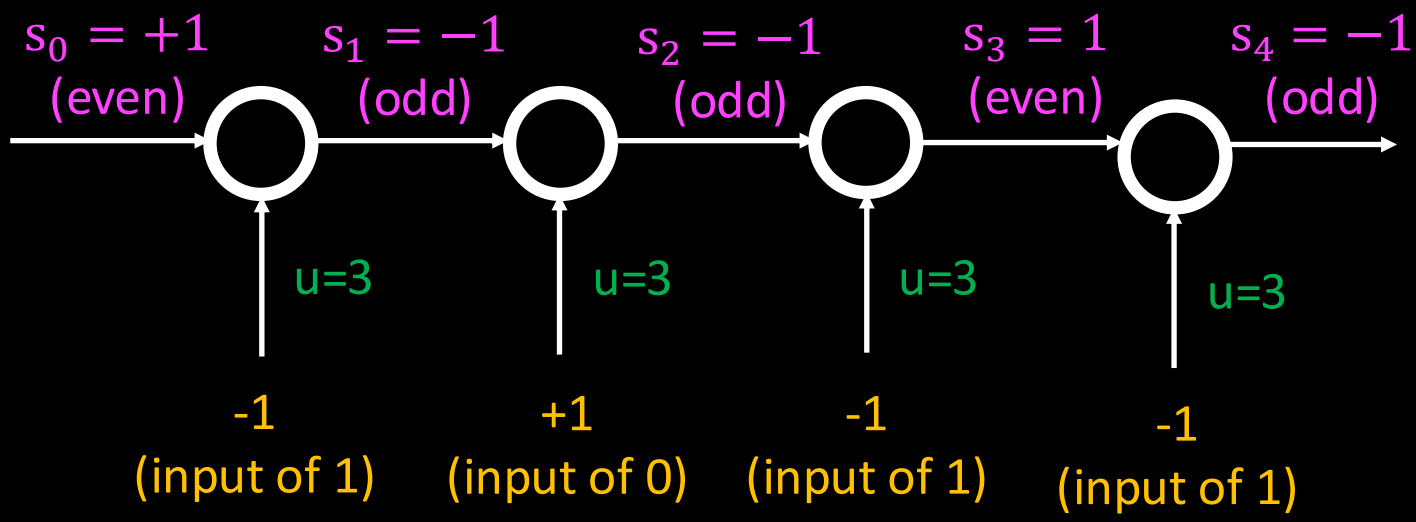




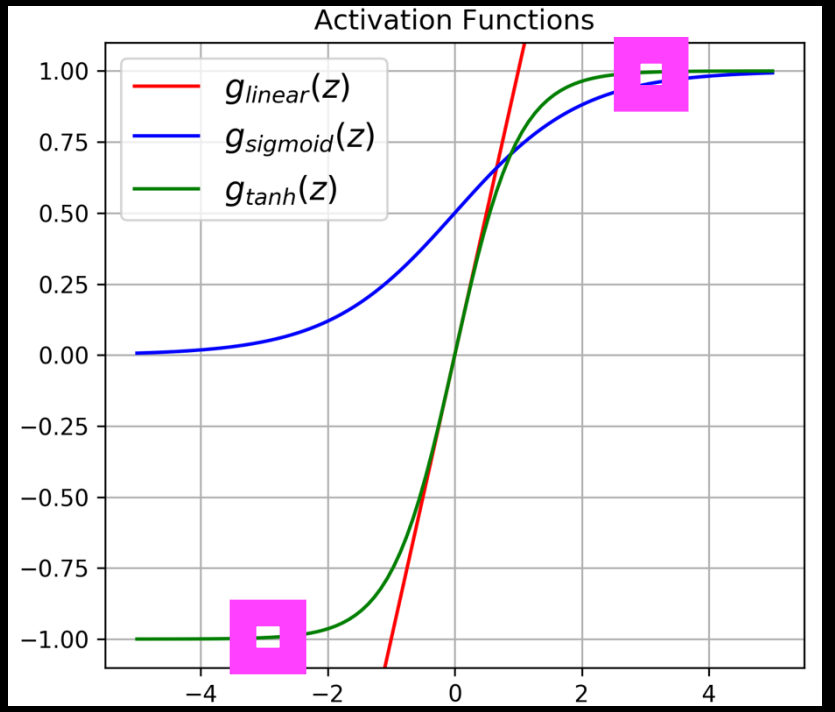
https://dustinstansbury.github.io/theclevermachine/derivation-common-neural-network-activation-functions

Example 1: Predict Parity

- What should happen to the memory state (s_t) to indicate if the number of 1s in sequence is even (+1) vs odd (-1)?
- Input of “1” encoded as “-1” and “0” encoded as +1



Novelty: the neural network has memory!



https://dustinstansbury.github.io/theclevermachine/derivation-common-neural-network-activation-functions

$$s_t = \tanh(u \cdot x_t \cdot s_{t-1})$$

$$s_1 = \tanh(3 \cdot -1 \cdot 1) \approx -1$$

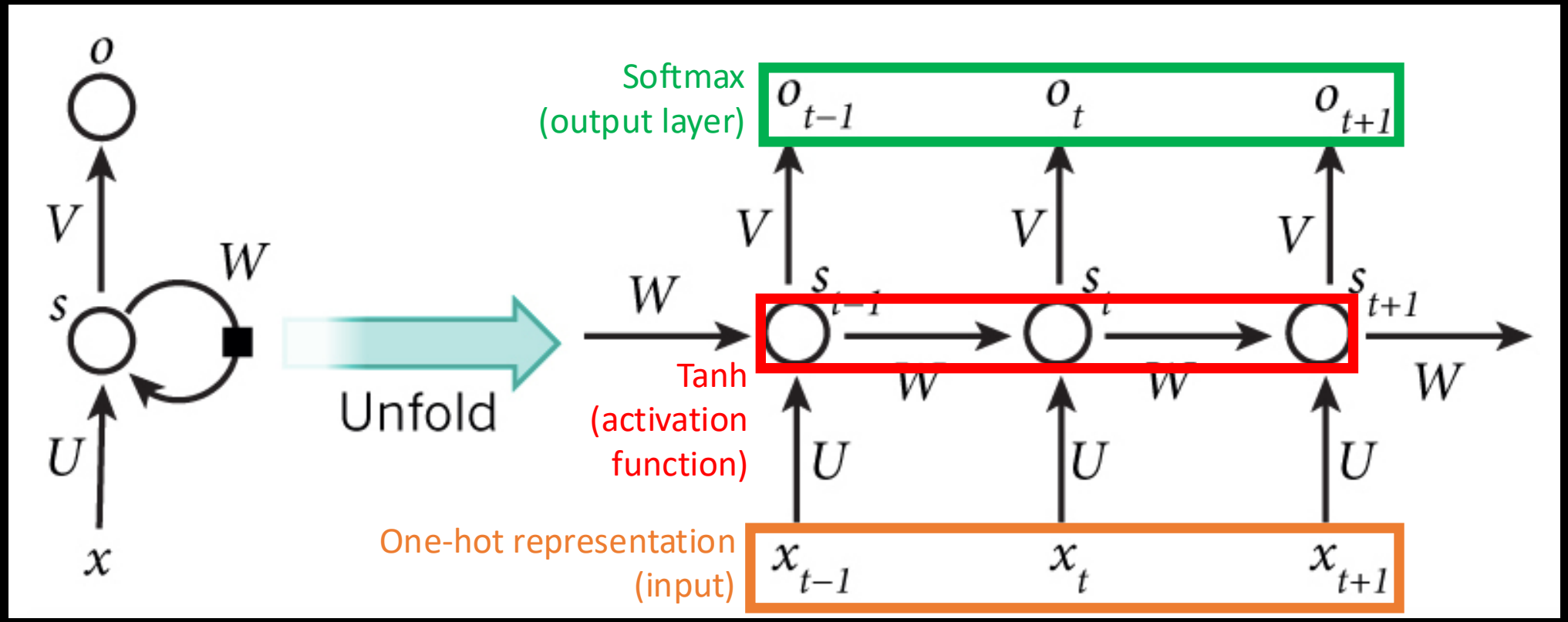
$$s_2 = \tanh(3 \cdot 1 \cdot -1) \approx -1$$

$$s_3 = \tanh(3 \cdot -1 \cdot -1) \approx 1$$

$$s_4 = \tanh(3 \cdot -1 \cdot 1) \approx -1$$

Example 2: Predict Next Character of Sequence

- Goal: predict next character in text (i.e., automatic text completion)

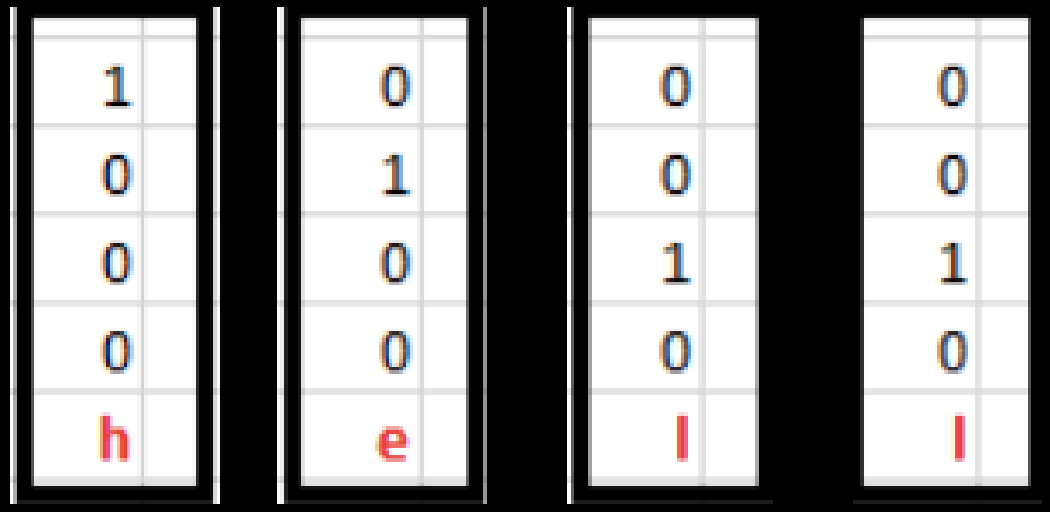


- Three new ingredients

http://www.wildml.com/2015/09/recurrent-neural-networks-tutorial-part-1-introduction-to-rnns/

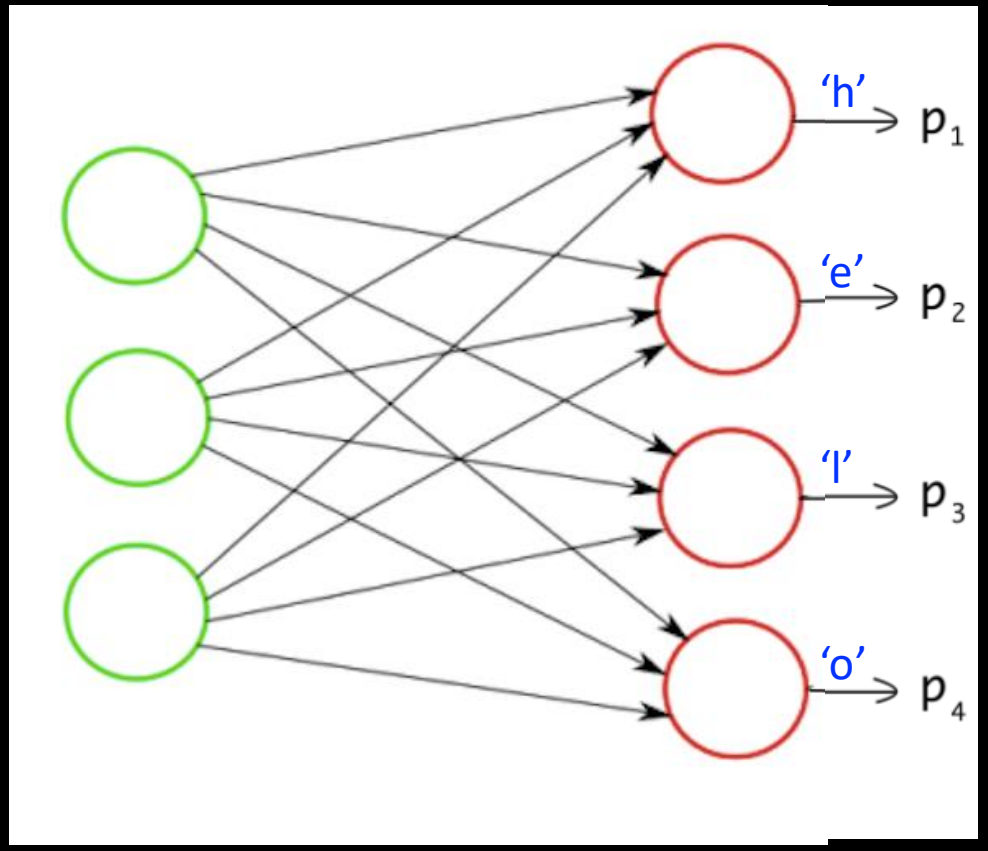
Example 2: Predict Next Character of Sequence

- One hot vector representation: convert text to numeric format; e.g.,
  1. Tokenize training data: “hello” -> “h”, “e”, “l”, “l”, “o”
  2. Learn vocabulary by identifying all unique tokens: {h, e, l, o}
  3. Encode data as vectors; e.g., one hot encoding
- e.g., for input sequence “hello”, what is the input at time step:
  - 1?
  - 2?
  - 3?
  - 4?
  - And so on…



Example 2: Predict Next Character of Sequence

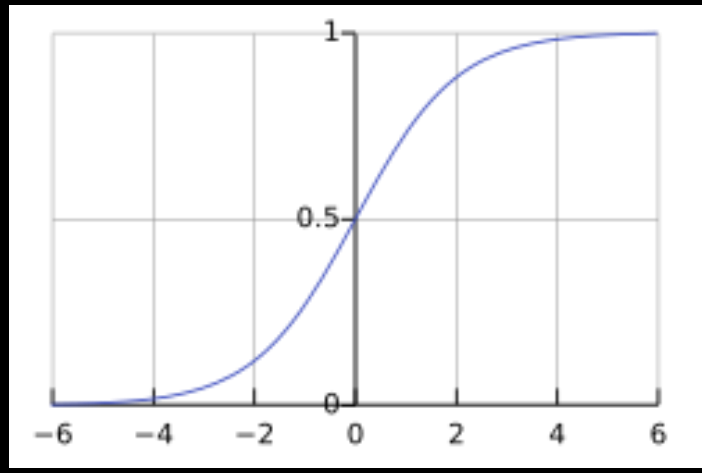
- Softmax (for output layer): why not just add sigmoid neurons at the output? e.g.,



- By independently predicting for each class, the model can predict multiple classes rather than choose exactly one from the multiple class options

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

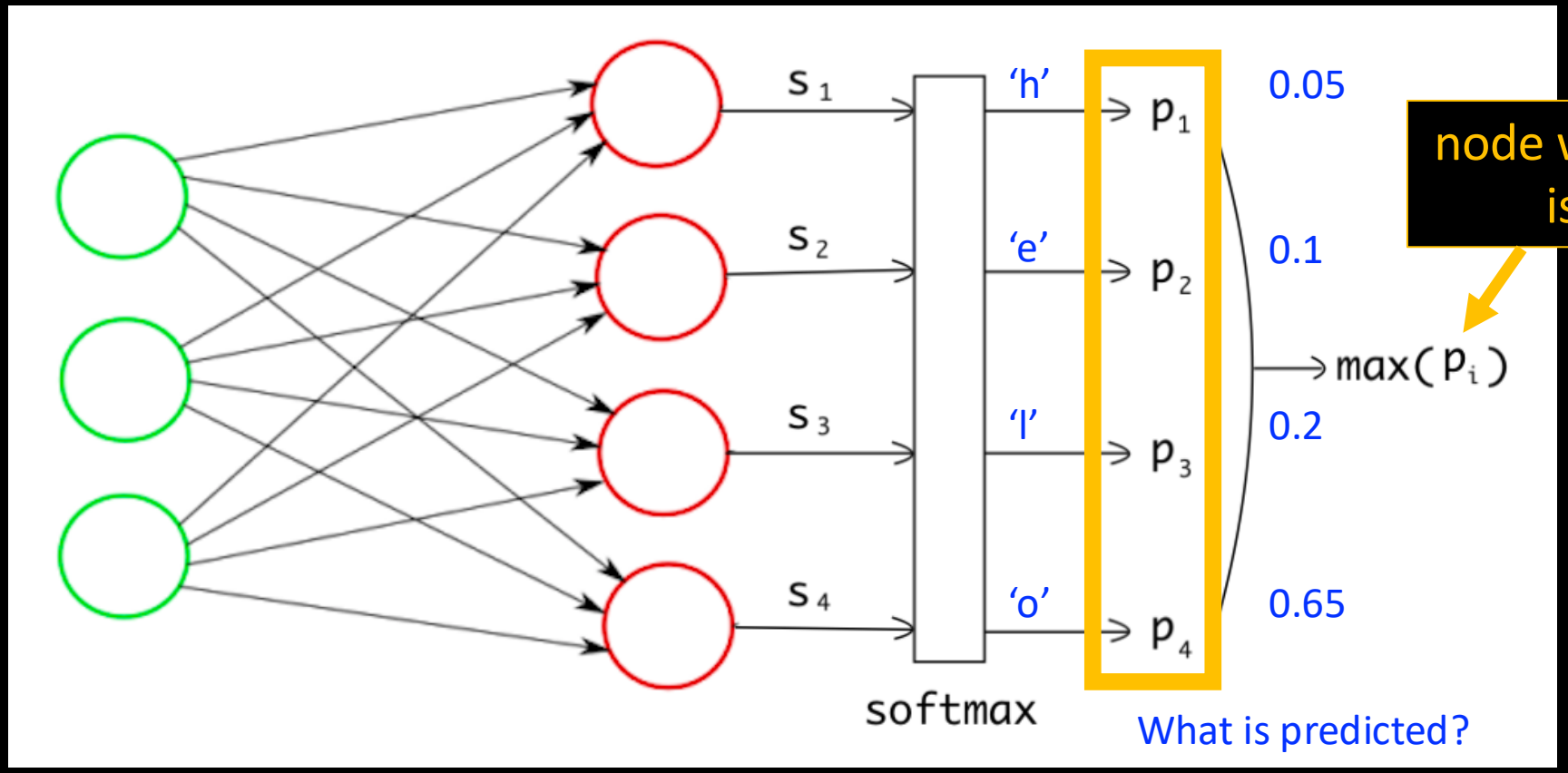
:



https://towardsdatascience.com/multi-label-image-classification-with-neural-network-keras-ddc1ab1afede

Example 2: Predict Next Character of Sequence

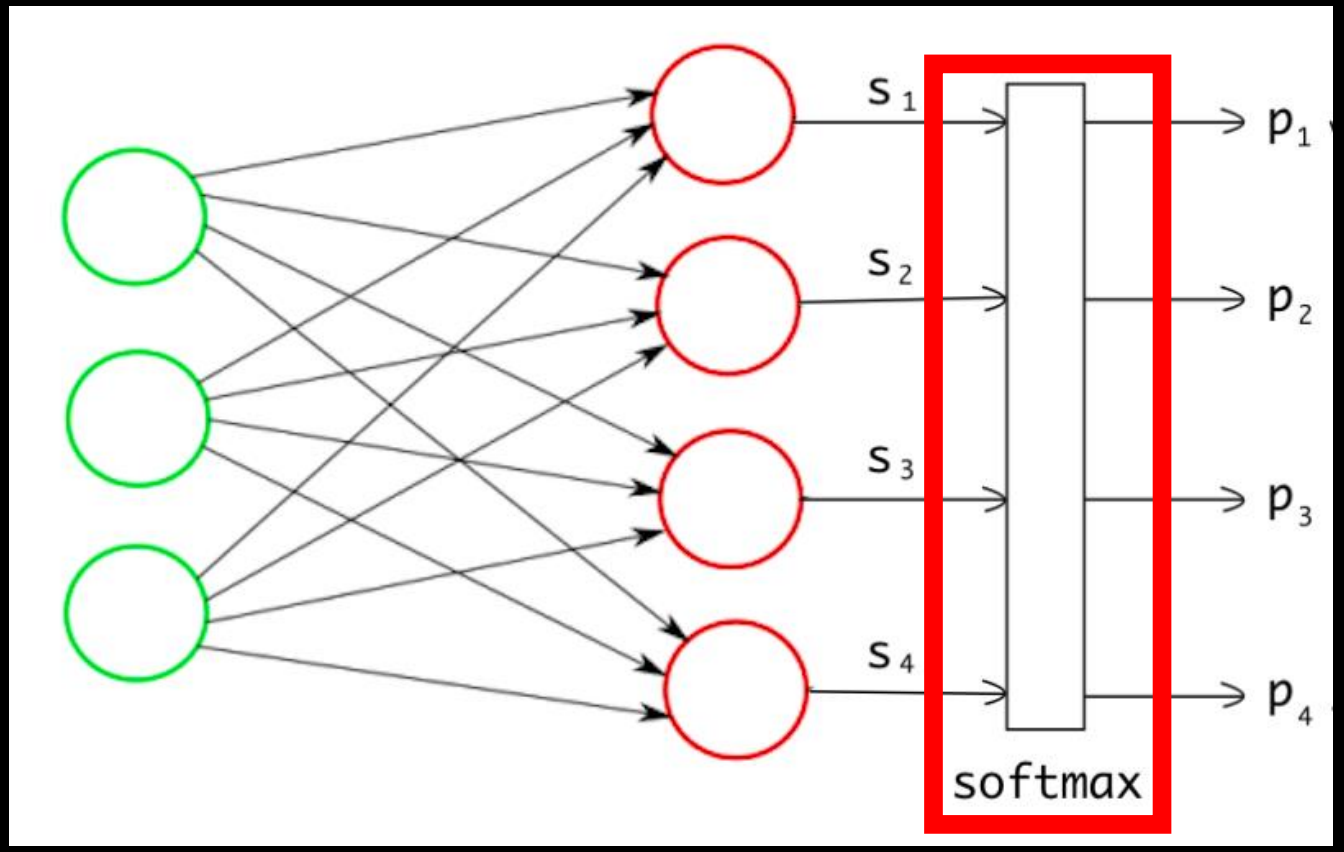
- Softmax (for output layer): generates probability distribution that sums to 1; e.g.,



https://towardsdatascience.com/multi-label-image-classification-with-neural-network-keras-ddc1ab1afede

Example 2: Predict Next Character of Sequence

- Softmax (for output layer): generates probability distribution that sums to 1; e.g.,



Makes incoming score positive while preserving its order across all scores

$$p_i = \frac{e^{s_i}}{\sum_{j=1}^{K} e^{s_j}}$$

Normalizes all class probabilities so they sum to 1; how many classes (K) is the model predicting from in this example?

https://towardsdatascience.com/multi-label-image-classification-with-neural-network-keras-ddc1ab1afede

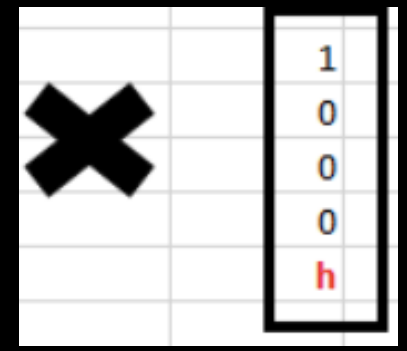
Example 2: Predict Next Character of Sequence

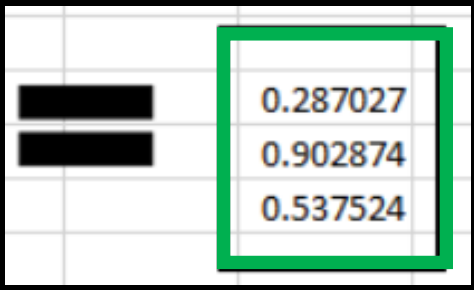
Initialize to random value: 0.567001

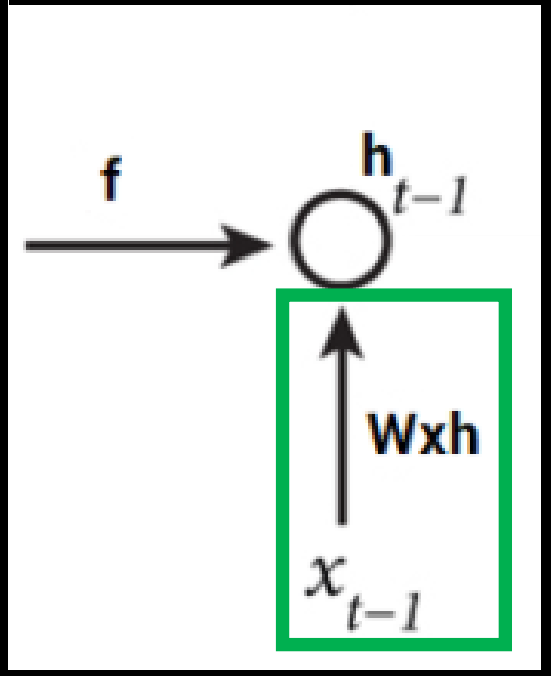
$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + \text{bias})$$

Initialize to 0

wxh			
0.287027	0.84606	0.572392	0.486813
0.902874	0.871522	0.691079	0.18998
0.537524	0.09224	0.558159	0.491528







The number of rows is a design choice; Why use three rows instead of 1 row for the weight matrix?

- more parameters can represent a more complex function

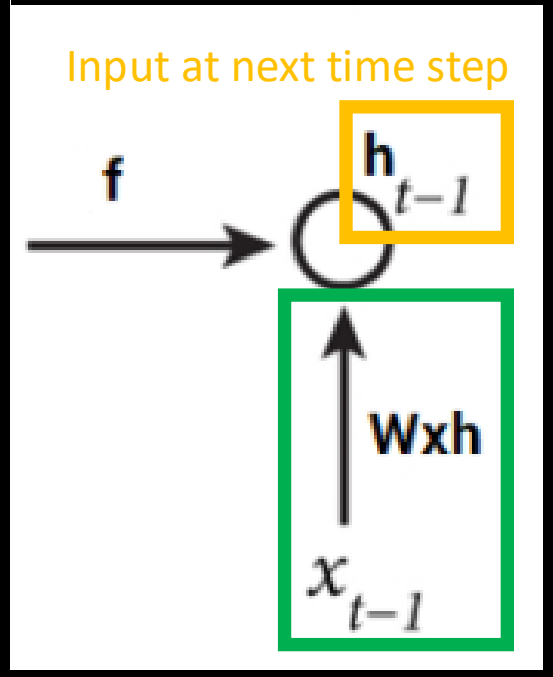
https://www.analyticsvidhya.com/blog/2017/12/introduction-to-recurrent-neural-networks/

Example 2: Predict Next Character of Sequence

Initialize to random value: 0.567001

$$h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + \text{bias})$$

Initialize to 0



wxh			
0.287027	0.84606	0.572392	0.486813
0.902874	0.871522	0.691079	0.18998
0.537524	0.09224	0.558159	0.491528

× [1, 0, 0, 0, h] = [0.287027, 0.902874, 0.537524]

H_t = TANH { [0.287027359, 0.902874425, 0.537523791] + [0.567001, 0.567001, 0.567001] }

= [0.693168, 0.899554, 0.802118]

https://www.analyticsvidhya.com/blog/2017/12/introduction-to-recurrent-neural-networks/

Example 2: Predict Next Character of Sequence

Initialize to random value: 0.427043

Recall: Initialized to random value: 0.567001

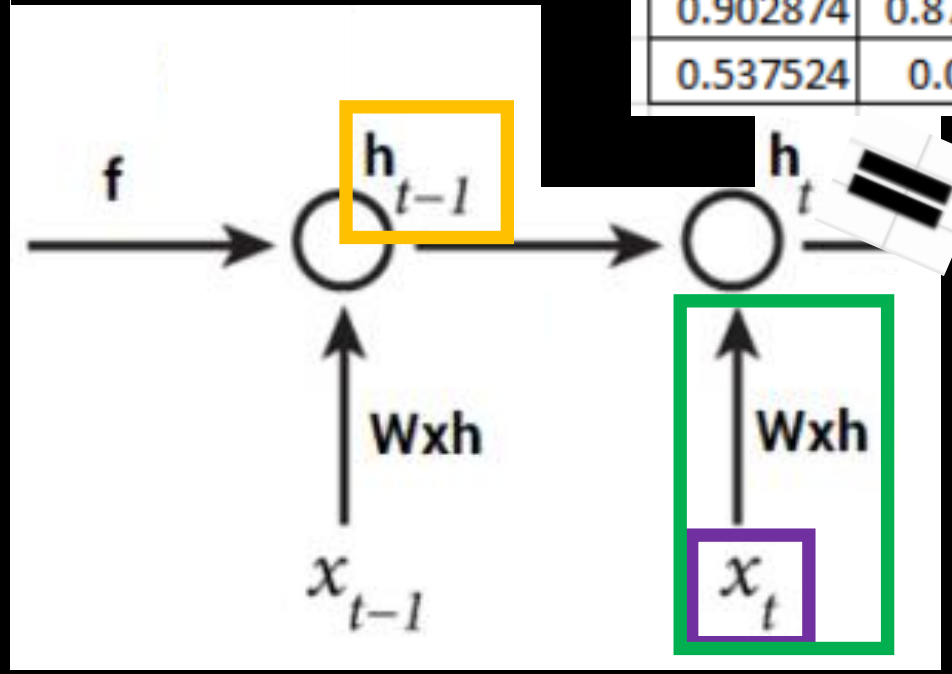
$$h_t = \tanh\left(W_{hh} h_{t-1} + W_{xh} x_t + \text{bias}\right)$$

Output at previous time step

wxh			
0.287027	0.84606	0.572392	0.486813
0.902874	0.871522	0.691079	0.18998
0.537524	0.09224	0.558159	0.491528

0
1
0
0
e

0.84606
0.871522
0.09224



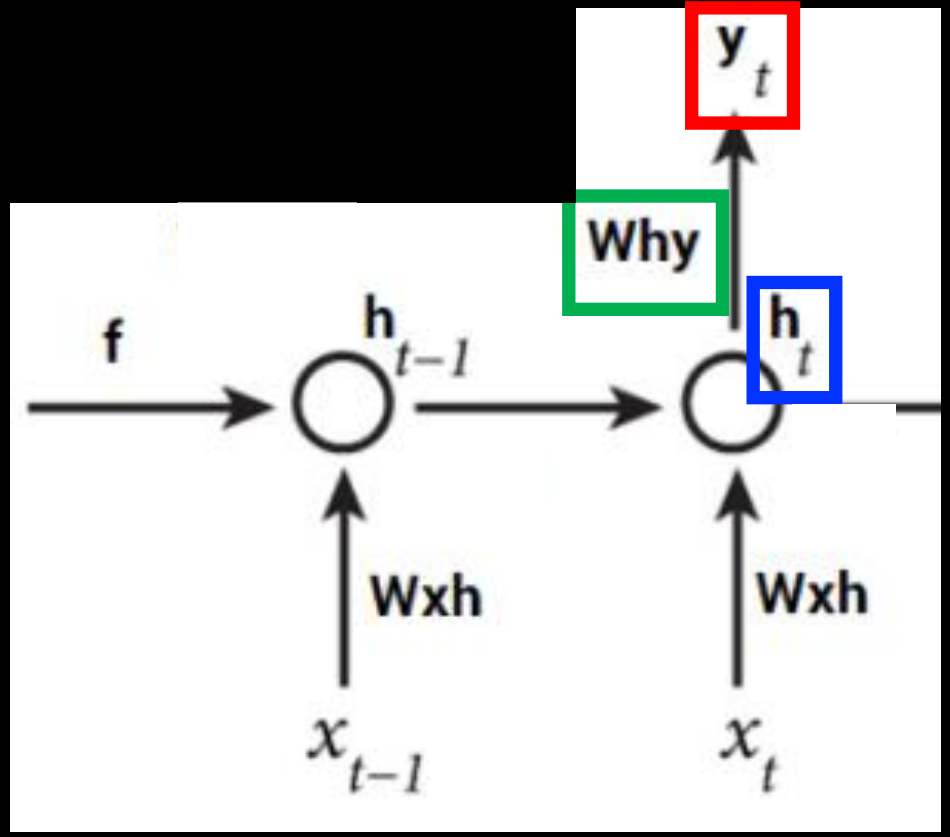
TANH { 0.427043 × (0.693168, 0.899554, 0.802118) + (0.84606, 0.871522, 0.09224) + (0.567001, 0.567001, 0.567001) }

0.93653372
0.94910403
0.76234056

https://www.analyticsvidhya.com/blog/2017/12/introduction-to-recurrent-neural-networks/

Example 2: Predict Next Character of Sequence

$y_t = W_{hy} h_t$



why		
0.37168	0.974829459	0.830034886
0.39141	0.282585823	0.659835709
0.64985	0.09821557	0.334287084
0.91266	0.32581642	0.144630018

×

Ht
0.936534
0.949104
0.762341

(assume bias is 0 for this example)

=

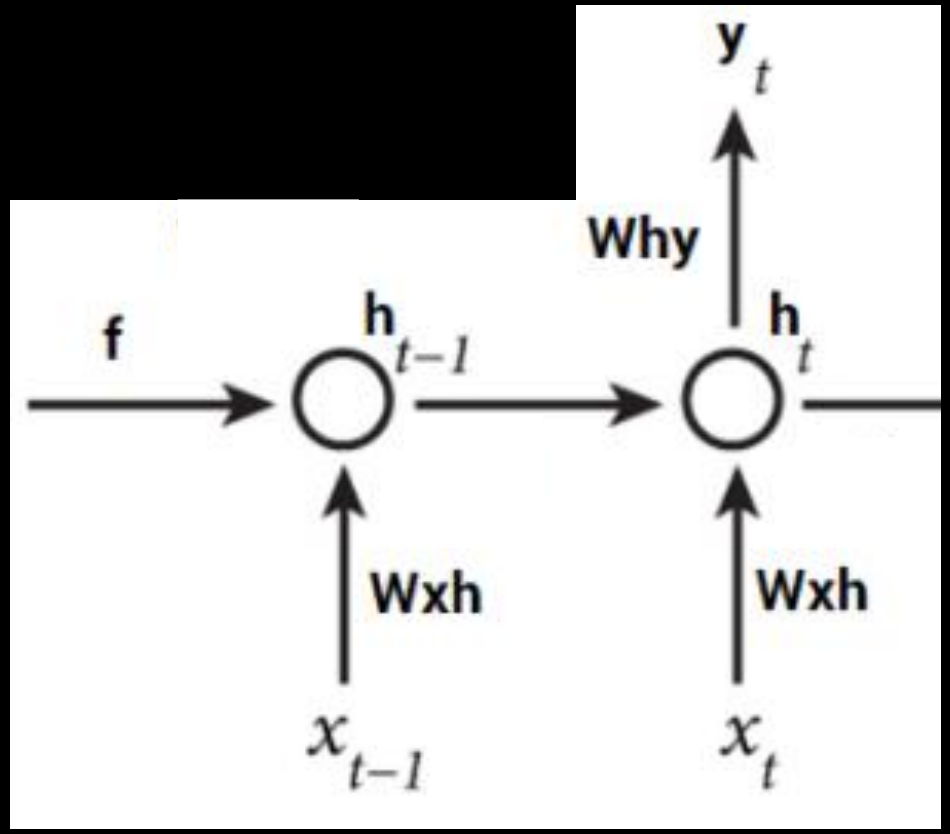
yt
1.90607732
1.13779113
0.95666016
1.27422602

Applying softmax, to compute letter probabilities:

0.419748
0.194682
0.162429
0.223141

https://www.analyticsvidhya.com/blog/2017/12/introduction-to-recurrent-neural-networks/

Example 2: Predict Next Character of Sequence



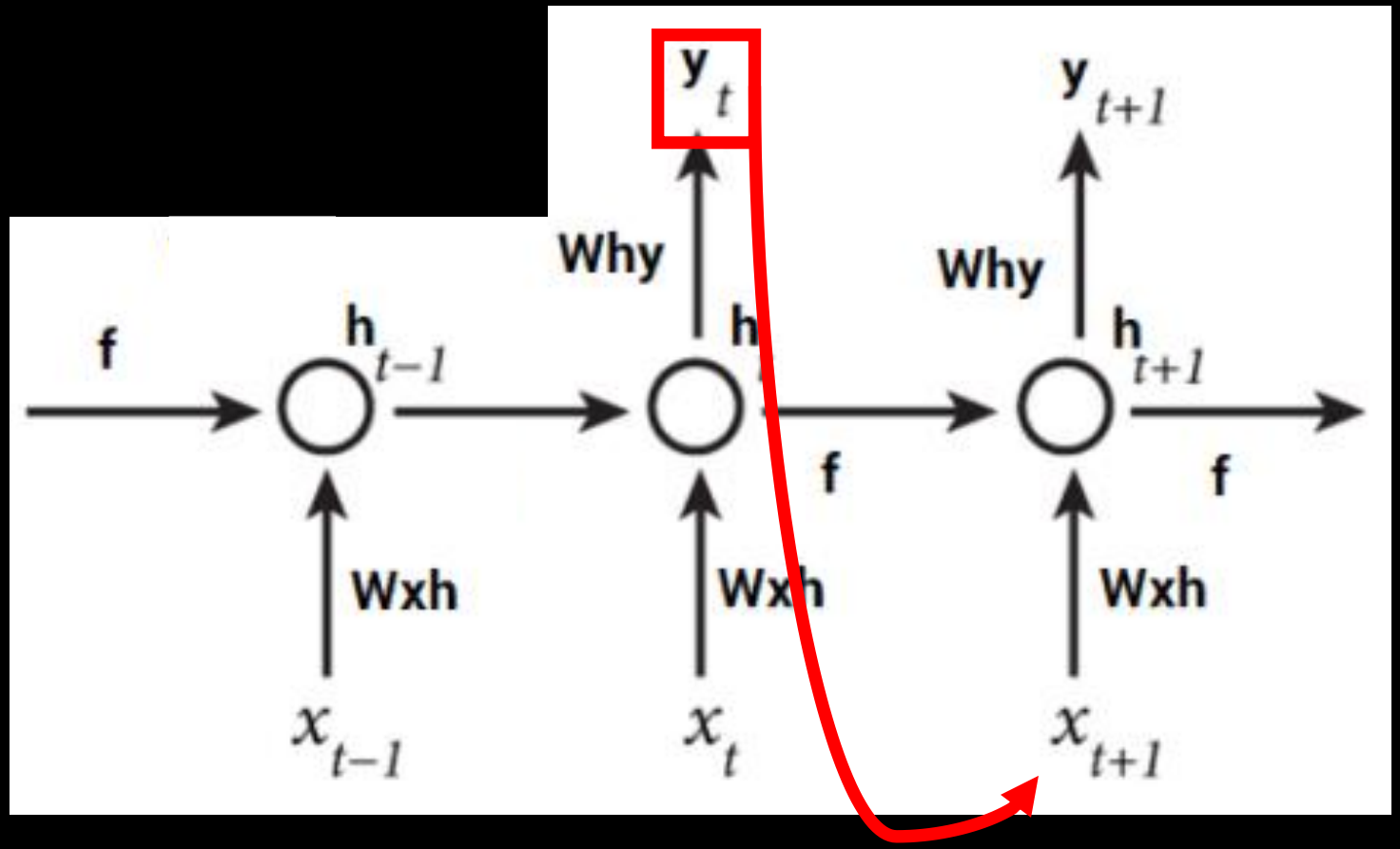
What letter is predicted from our vocabulary {h, e, l, o}?

Applying softmax, to compute letter probabilities:

0.419748
0.194682
0.162429
0.223141

https://www.analyticsvidhya.com/blog/2017/12/introduction-to-recurrent-neural-networks/

Example 2: Predict Next Character of Sequence



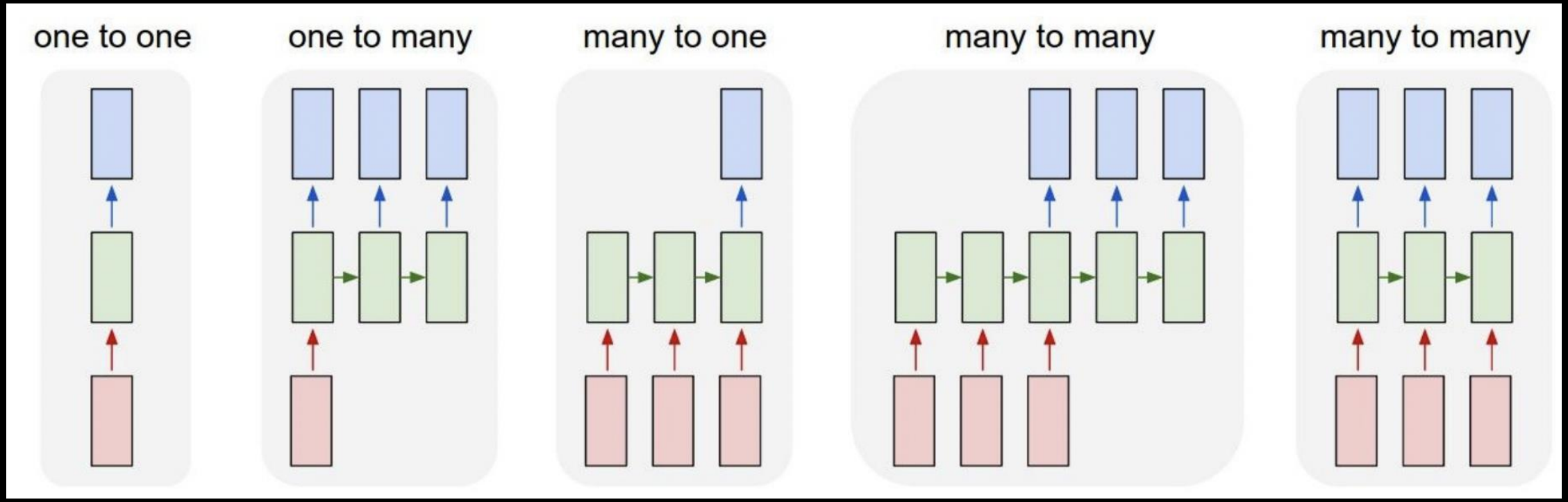
...

Feed previous prediction as input for next time step

When to stop predicting sequence?
e.g.,
- first predict length to decide
- predict special output symbol

https://www.analyticsvidhya.com/blog/2017/12/introduction-to-recurrent-neural-networks/

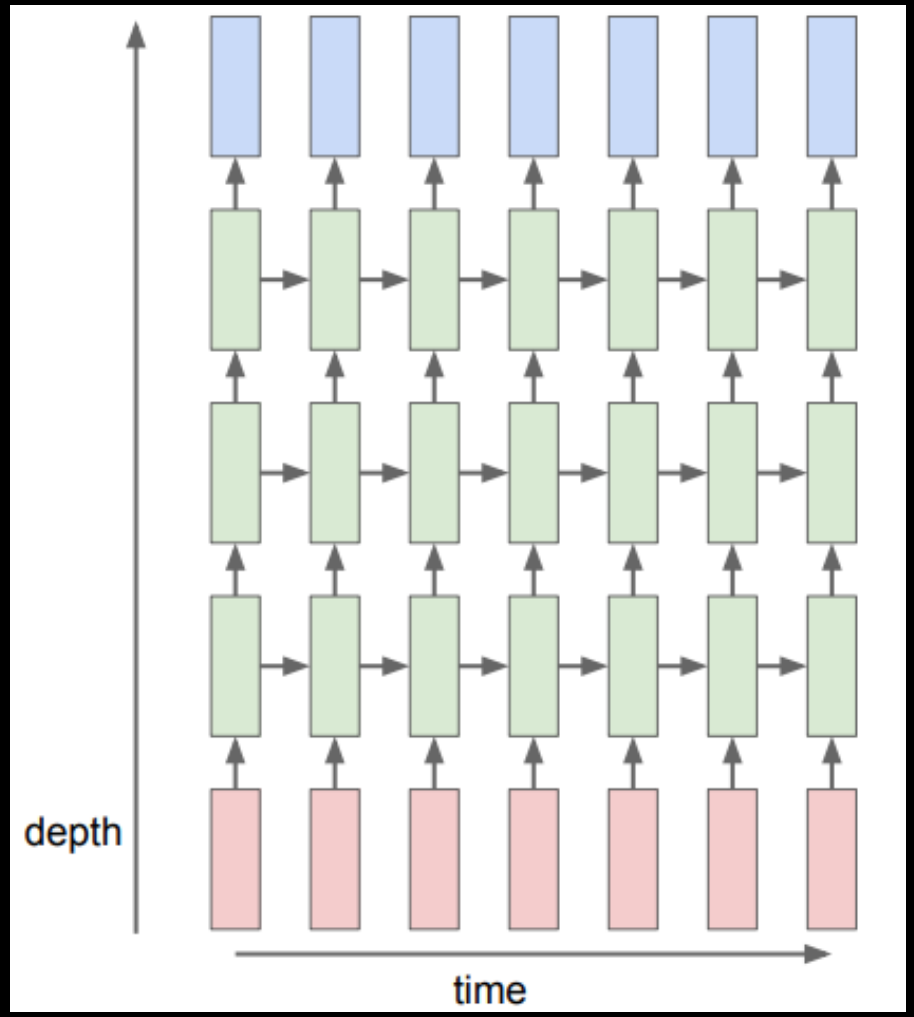
RNN Variants: Variable Input/Output Lengths



Breakout discussion questions:

1. Which variant(s) would you use for text classification and why?
2. When and why choose the first variant of “many to many” versus the second?

RNN Variants: Can Choose Different Number of Hidden Layers and Number of Nodes Per Layer



http://cs231n.stanford.edu/slides/2016/winter1516_lecture10.pdf

RNN Training: Backpropagation Through Time

- Repeat until stopping criterion met:

1. Propagate training data through unfolded network to predict output

2. Measure error of predictions using a loss function

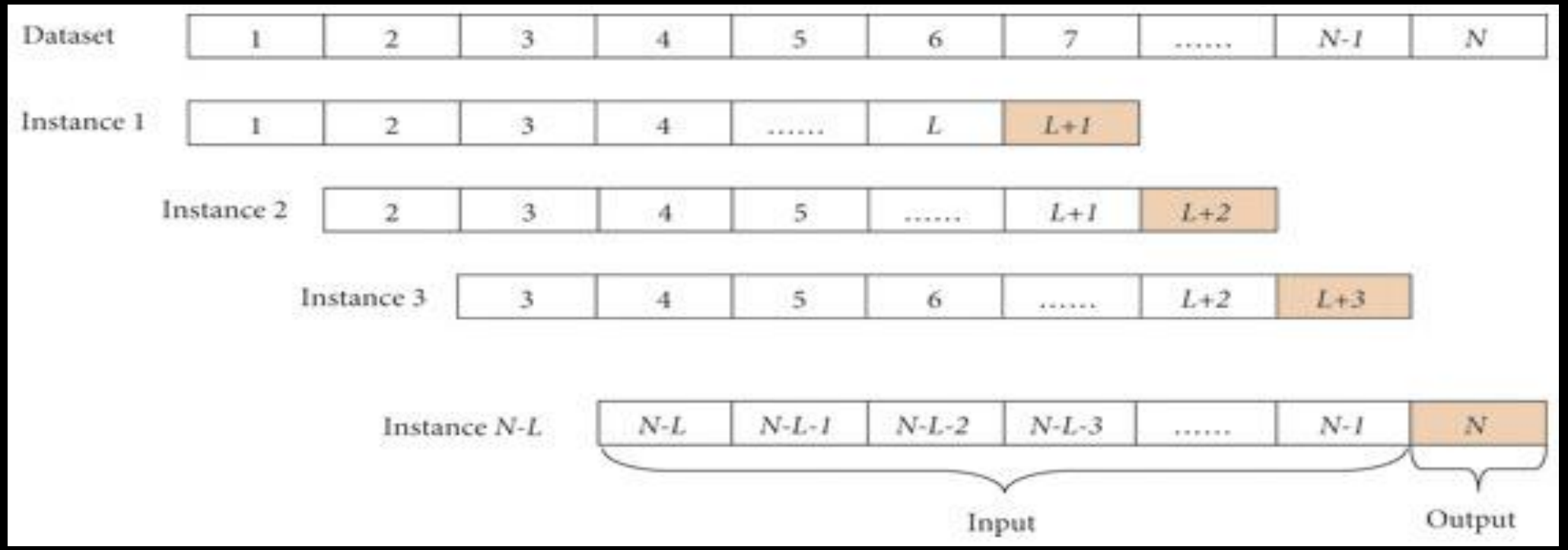
How to support sequence inputs?

3. Calculate gradients of the loss function to determine how each model parameter contributed to the error; weight sharing accounted for by summing gradients across all time steps

4. Update each parameter using calculated gradients

RNN Training: Supporting Variable Input Length

- A priori, set (1) window size and (2) step size
  - For longer sequences, truncate to window size L



  - For shorter sequences, pad with zeros

https://onlinelibrary.wiley.com/doi/10.1155/2022/5596676

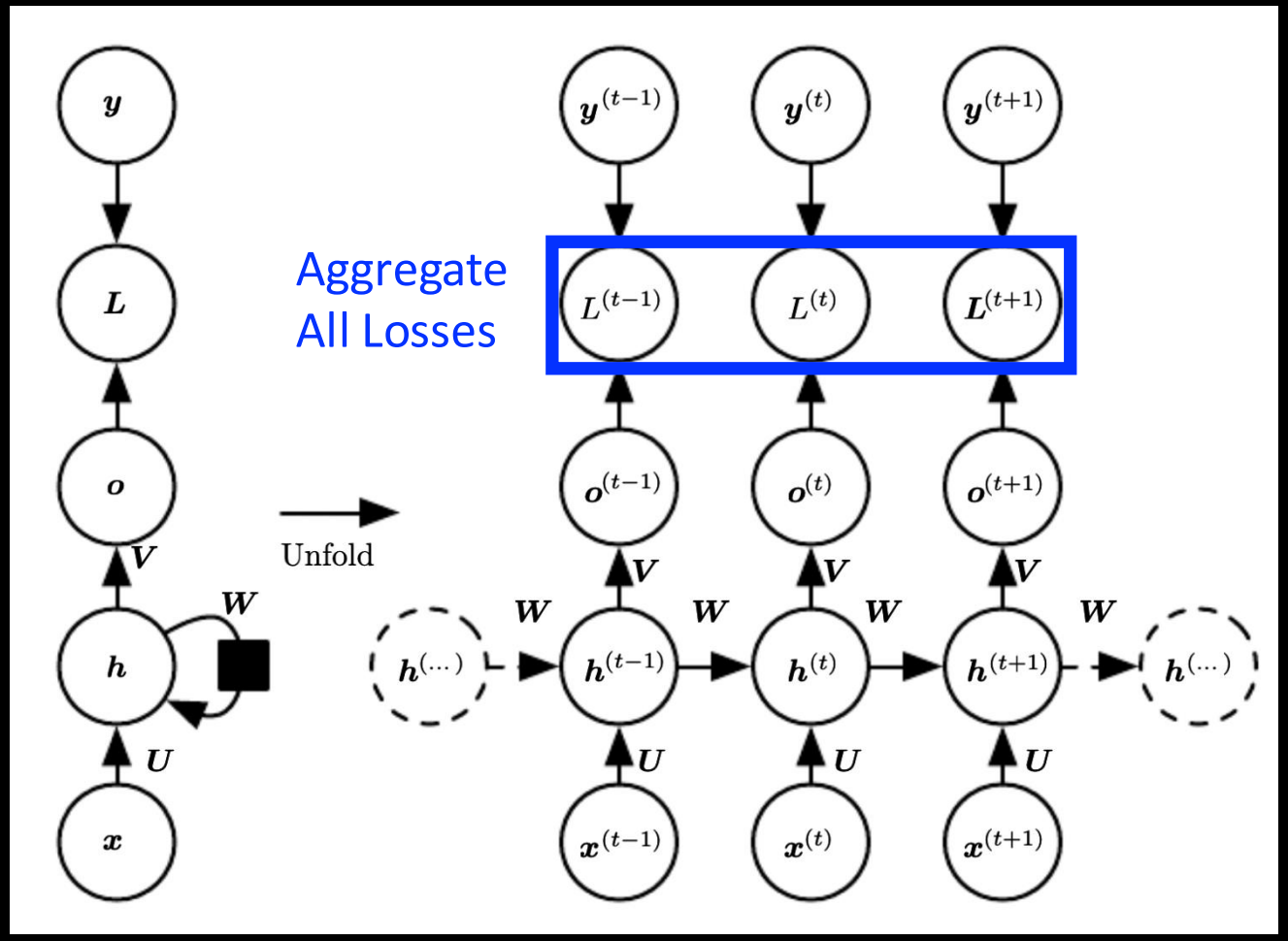
RNN Training: Backpropagation Through Time

- Repeat until stopping criterion met:

1. Propagate training data through unfolded network to predict output
2. Measure error of predictions using a loss function
3. Calculate gradients of the loss function to determine how each model parameter contributed to the error; weight sharing accounted for by summing gradients across all time steps
4. Update each parameter using calculated gradients

How to handle when sequences are predicted?

RNN Training: How to Handle “N-to-Many”?



Goodfellow, Bengio, and Courville, Deep Learning.

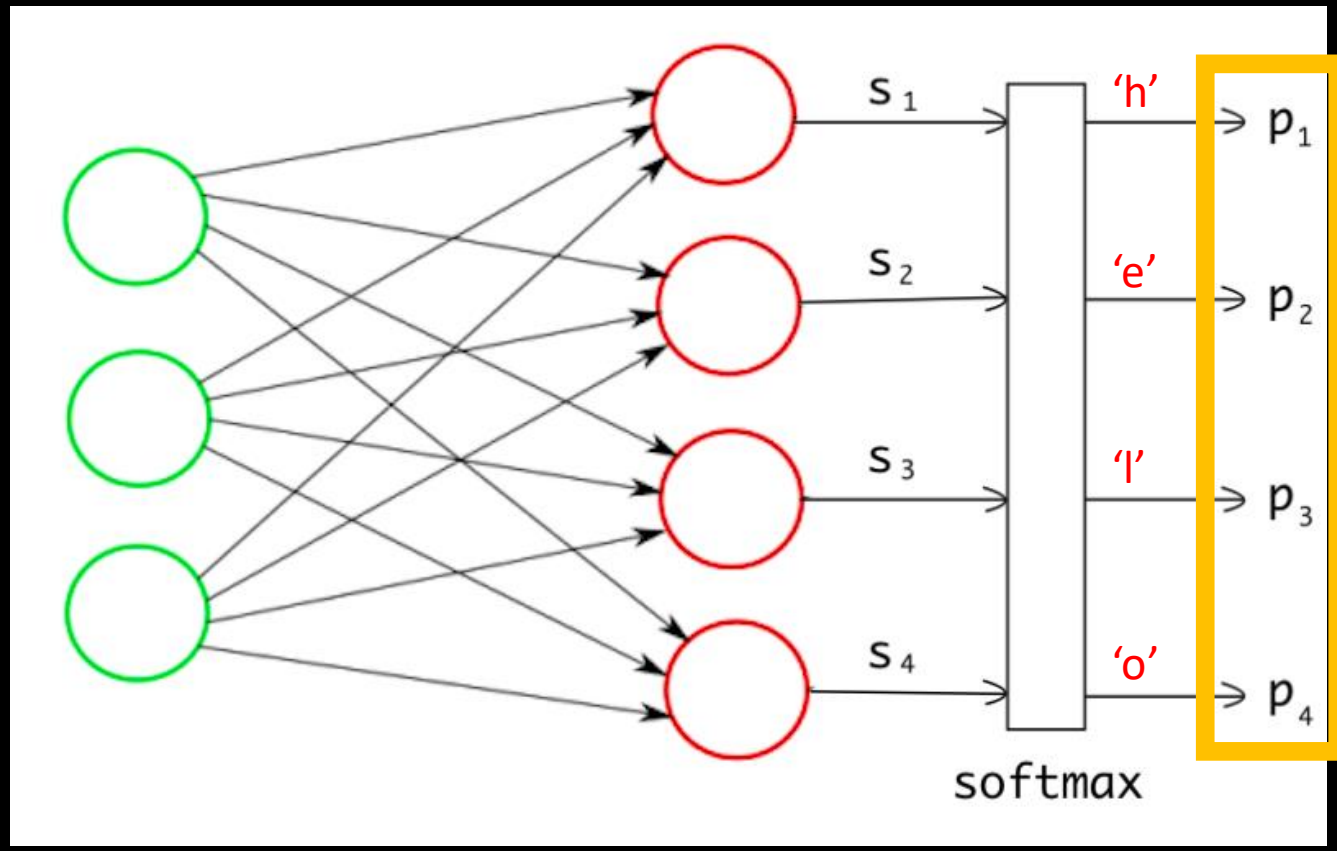
RNN Training: Backpropagation Through Time

- Repeat until stopping criterion met:

    1. Propagate training data through unfolded network to predict output
    2. Measure error of predictions using a loss function
    3. Calculate gradients of the loss function to determine how each model parameter contributed to the error; weight sharing accounted for by summing gradients across all time steps
    4. Update each parameter using calculated gradients

What should be the loss function for multi-class classification? (e.g., predict character)

Multi-Class Classification Learning



Target: model parameters that predict the true class distribution of 1 hot vector (i.e., one 1 and the rest 0s)

https://towardsdatascience.com/multi-label-image-classification-with-neural-network-keras-ddc1ab1afede

Multi-Class Classification Learning: Cross Entropy Loss Function

Probability distribution of true class

Probability distribution of predicted class

Number of classes

$$L_{\text{CE}}(\hat{y}, y) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

Recall, truth is set to 1 for one class and 0 otherwise

$$= -\log \hat{y}_k, \quad \text{(where } k \text{ is the correct class)}$$

Simplifies to the log of the predicted probability for the correct class (i.e., negative log likelihood loss)

Implementation detail: natural log (ln) is used rather than base 10 to improve gradient behavior for learning

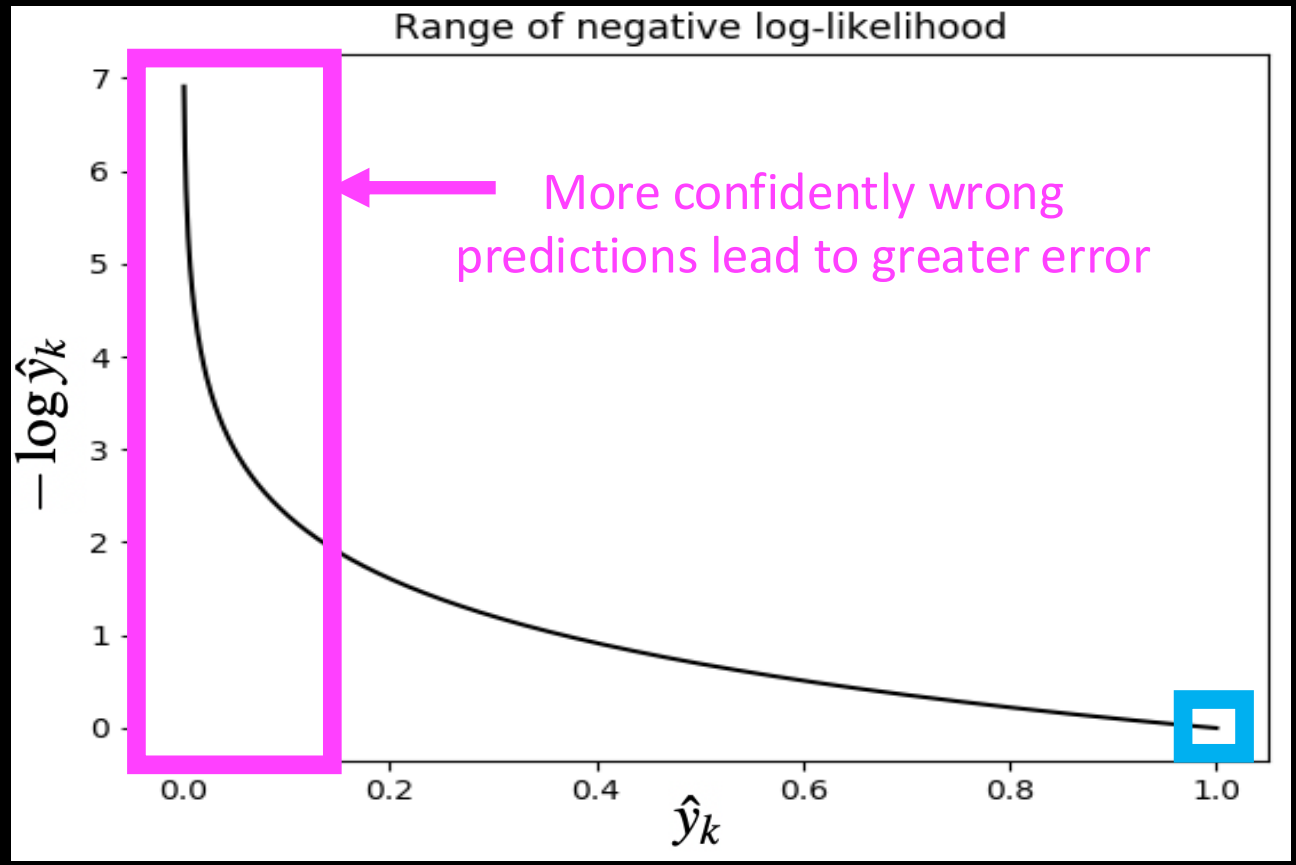
Excellent background: https://web.stanford.edu/~jurafsky/slp3/5.pdf

Multi-Class Classification Learning: Cross Entropy Loss Function

Do we want to increase or reduce this loss when modifying the model parameters during training?

What is the range of possible values?

- Minimum: 0 (-ln 1)
  - i.e., correct prediction
- Maximum: infinity (-ln 0)
  - i.e., incorrect prediction



https://ljvmiranda921.github.io/notebook/2017/08/13/softmax-and-the-negative-log-likelihood/

RNN Training: This Often Fails ☹; Why?

- Repeat until stopping criterion met:

    1. Propagate training data through unfolded network to predict output

    2. Measure error of predictions using a loss function

    3. Calculate gradients of the loss function to determine how each model parameter contributed to the error; weight sharing accounted for by summing gradients across all time steps

    4. Update each parameter using calculated gradients

Today's Topics

- Limitation of feedforward neural networks: supporting sequential data
- Key insight: *recurrent* neural networks (RNNs)
- Breakthrough: gated RNNs for dynamically deciding what to remember
- Programming tutorial: recurrent neural networks in PyTorch

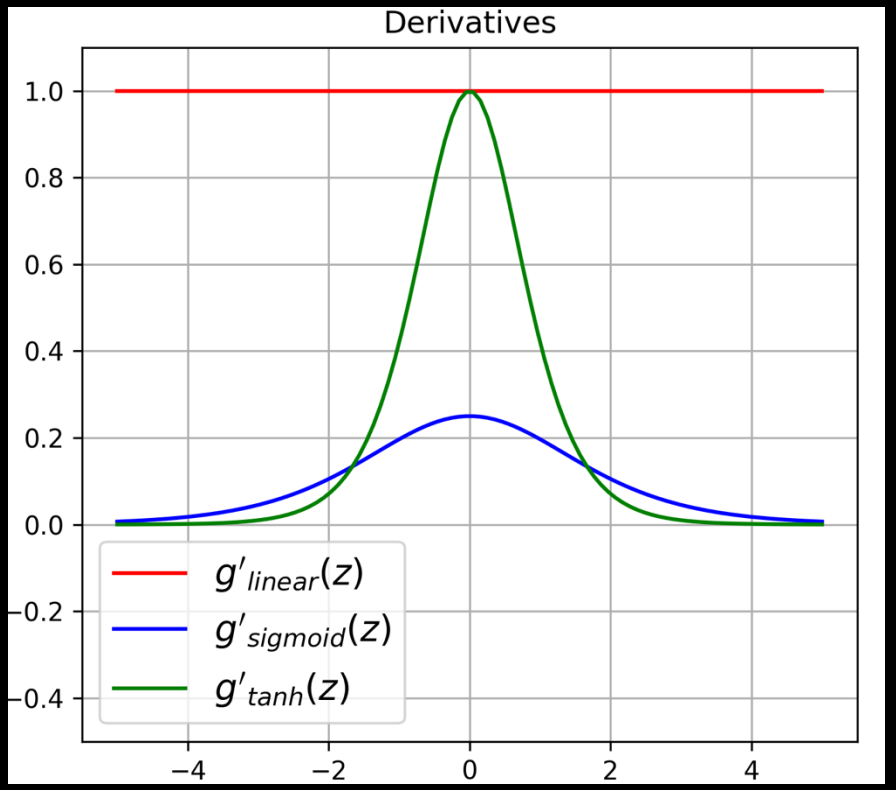
IEEE TRANSACTIONS ON NEURAL NETWORKS, VOL. 5, NO. 2, MARCH 1994

Learning Long-Term Dependencies with Gradient Descent is Difficult

Yoshua Bengio, Patrice Simard, and Paolo Frasconi, *Student Member, IEEE*

Key observation: derivative often less than 1

Why is this a problem?



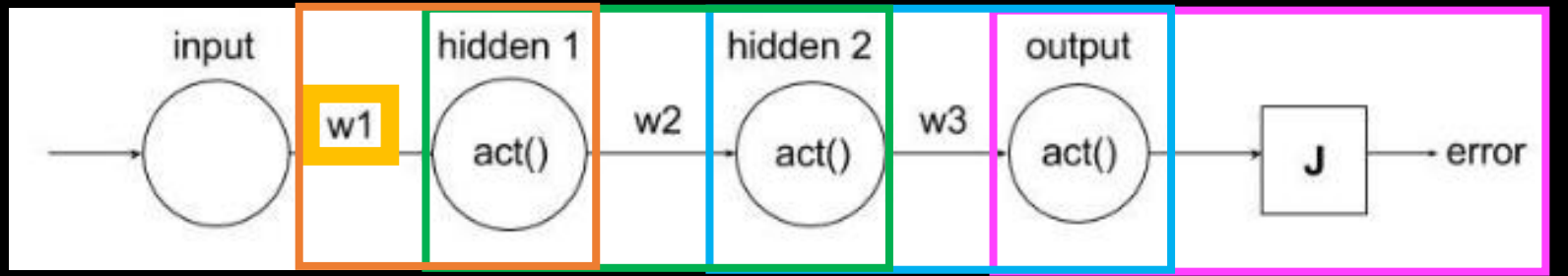
https://dustinstansbury.github.io/theclevermachine/derivation-common-neural-network-activation-functions

IEEE TRANSACTIONS ON NEURAL NETWORKS, VOL. 5, NO. 2, MARCH 1994

Learning Long-Term Dependencies with Gradient Descent is Difficult

Yoshua Bengio, Patrice Simard, and Paolo Frasconi, *Student Member, IEEE*

Backpropagation with toy example (unfolded):



Error derivative with respect to weight w1 (recall chain rule):

$$\frac{\partial error}{\partial w1} = \frac{\partial error}{\partial output} * \frac{\partial output}{\partial hidden2} * \frac{\partial hidden2}{\partial hidden1} * \frac{\partial hidden1}{\partial w1}$$

e.g., With sigmoid activation function: 0.25 * 0.25 * 0.25 * 0.25 = 0.004

Concern: learning signal decays exponentially with backpropagation, causing “long-term” past events to be effectively ignored (only recent events have sufficiently large gradients for learning)!

https://ayearofai.com/rohan-4-the-vanishing-gradient-problem-ec68f76ffb9b

IEEE TRANSACTIONS ON NEURAL NETWORKS, VOL. 5, NO. 2, MARCH 1994

Learning Long-Term Dependencies with Gradient Descent is Difficult

Yoshua Bengio, Patrice Simard, and Paolo Frasconi, *Student Member, IEEE*

Vanishing gradients analogy:

Snowplowing works locally, with areas cleared long ago already buried again



https://www.timesunion.com/news/article/albany-saratoga-snow-totals-16826908.php

Concern: learning signal decays exponentially with backpropagation, causing "long-term" past events to be effectively ignored (only recent events have sufficiently large gradients for learning)!

LONG SHORT-TERM MEMORY

Neural Computation 9(8):1735–1780, 1997

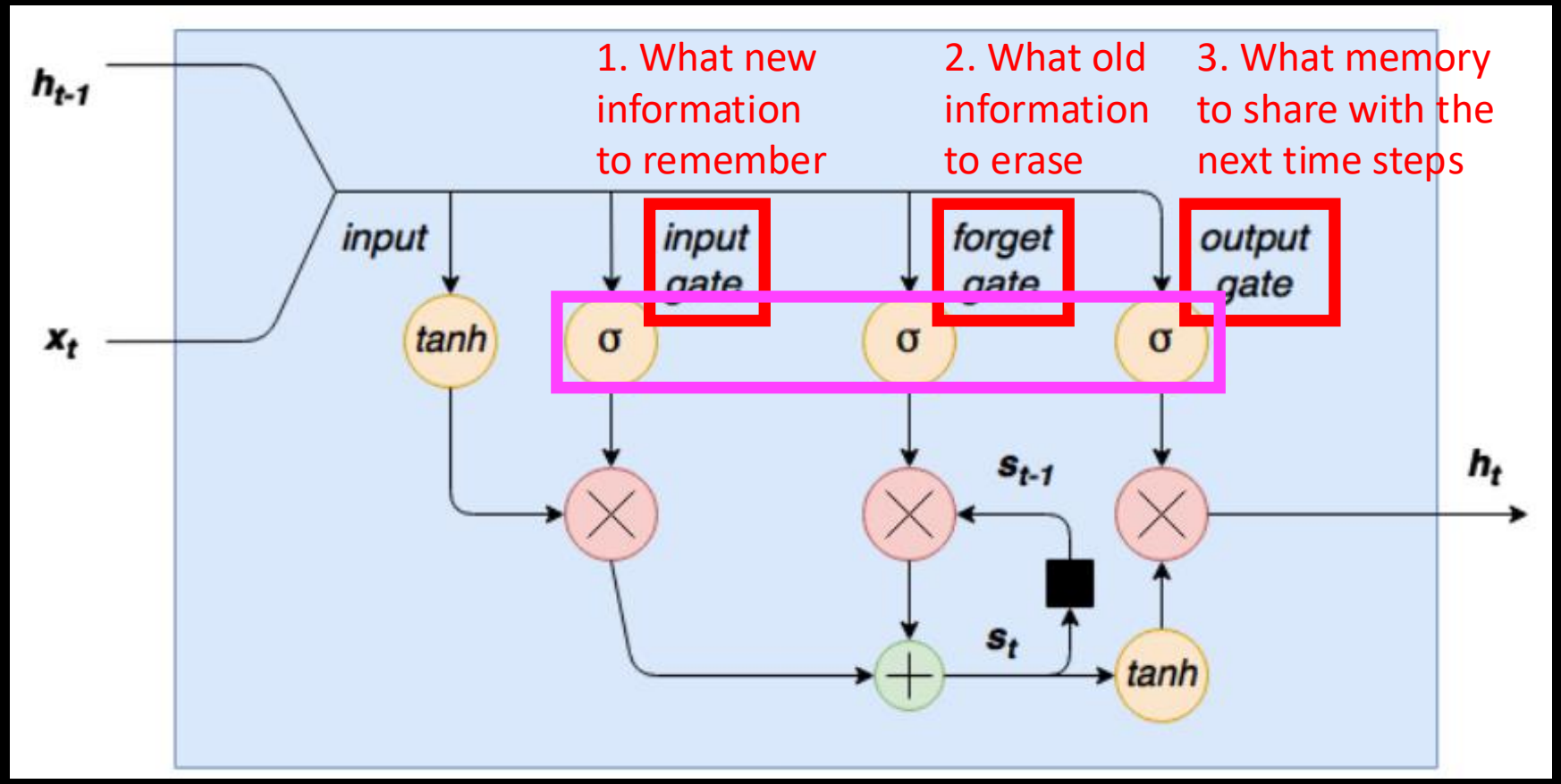
Sepp Hochreiter
Fakultät für Informatik
Technische Universität München
80290 München, Germany
hochreit@informatik.tu-muenchen.de
http://www7.informatik.tu-muenchen.de/~hochreit

Jürgen Schmidhuber
IDSIA
Corso Elvezia 36
6900 Lugano, Switzerland
juergen@idsia.ch
http://www.idsia.ch/~juergen

Idea: introduce architectural changes that prevent vanishing gradients!

LSTMs: Dynamically Decides What to Remember

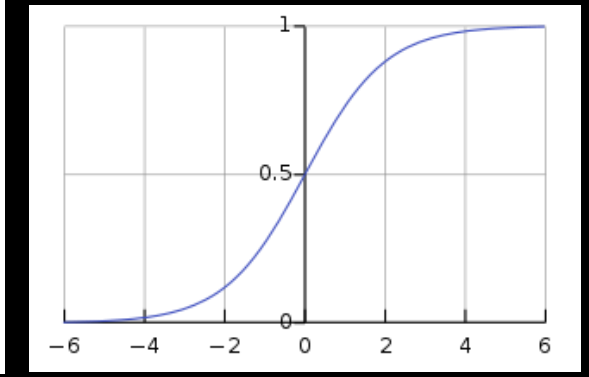
- Mechanism: adds 3 gates whose behaviors depend on the **current input** and **previous hidden state**



If "input" gate = 0?
- Discards new information

If "input" gate = 1?
- Keeps all new information

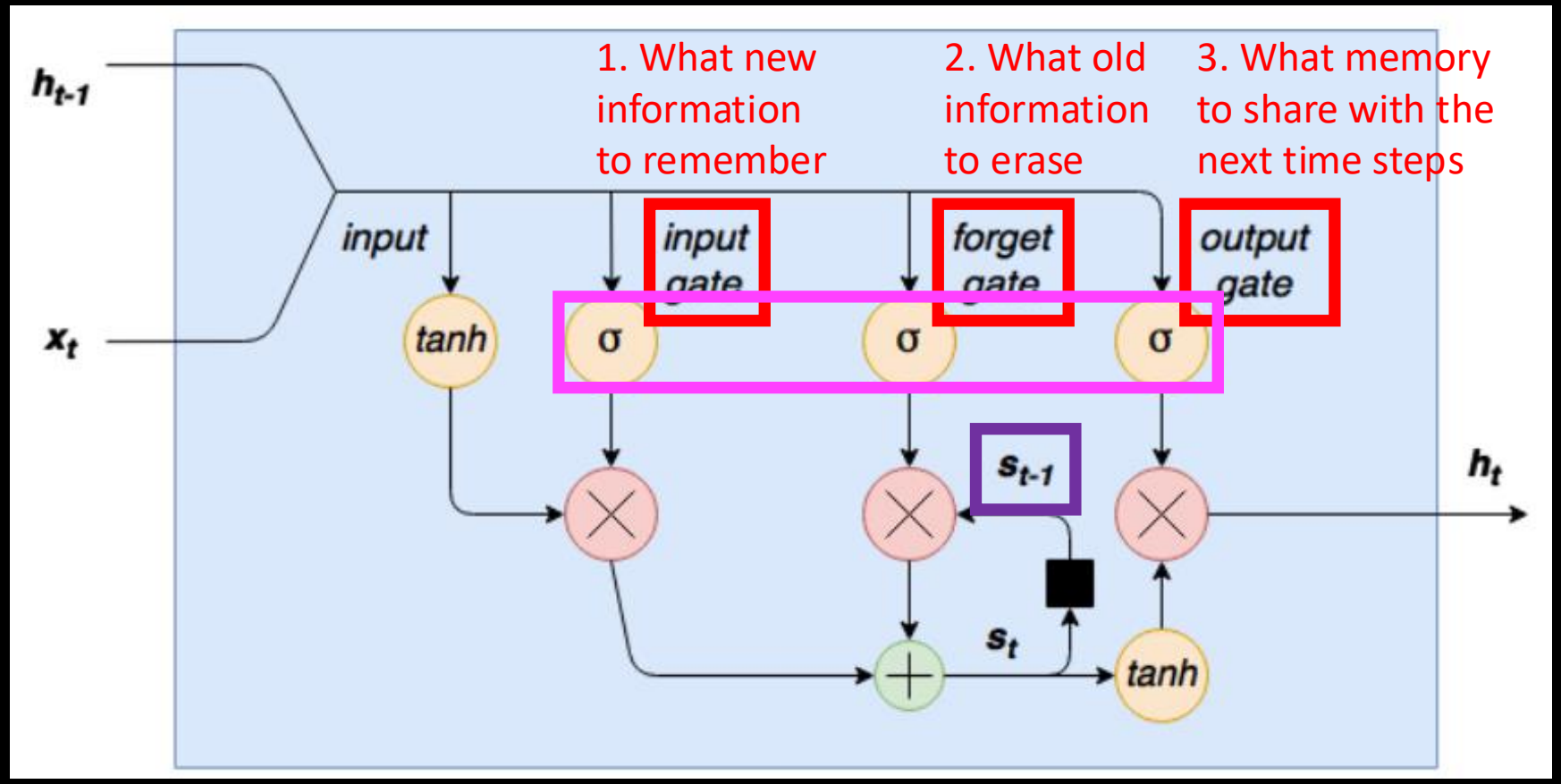
Called "gate" because the sigmoid function mimics open (1) and closed (0)



http://adventuresinmachinelearning.com/recurrent-neural-networks-lstm-tutorial-tensorflow/

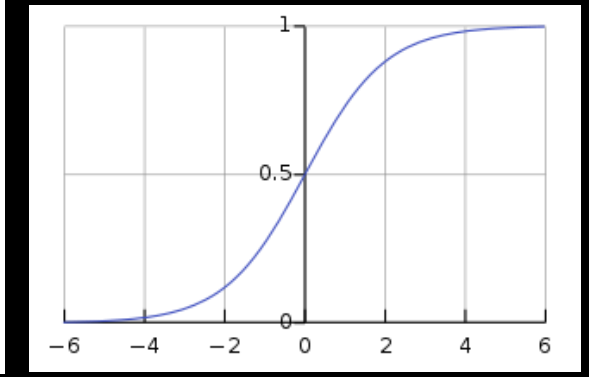
LSTMs: Dynamically Decides What to Remember

- Mechanism: adds 3 gates whose behaviors depend on the **current input** and **previous hidden state**



If "forget" gate = 0?
- Discards old information
If "forget" gate = 1?
- Keeps all old information

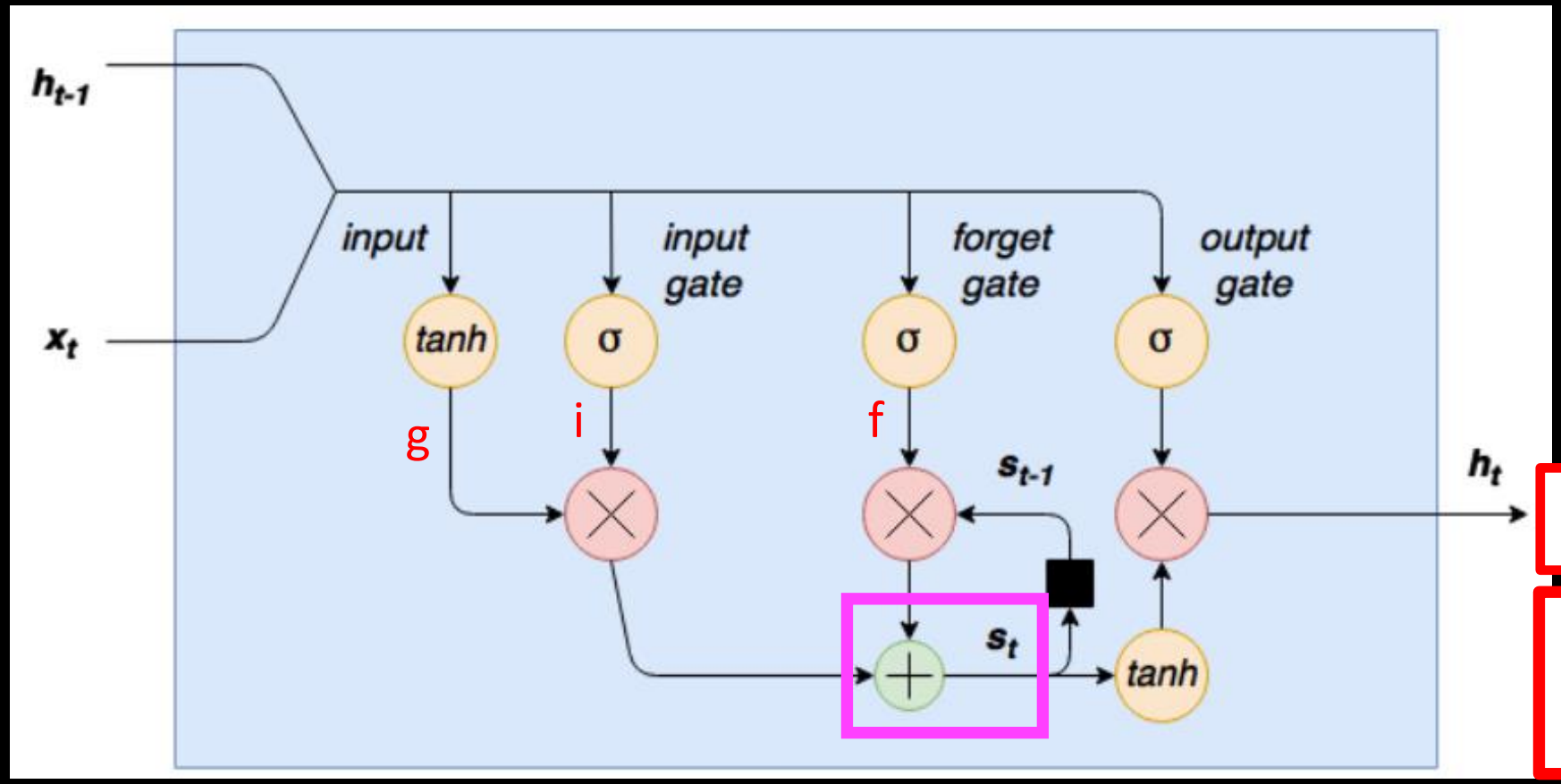
Called "gate" because the sigmoid function mimics open (1) and closed (0)



http://adventuresinmachinelearning.com/recurrent-neural-networks-lstm-tutorial-tensorflow/

LSTMs: Dynamically Decides What to Remember

- Longer-term memory support: possible because forget gate enables long-term learning



Addition enables the cell state to flow back through time without vanishing for long durations

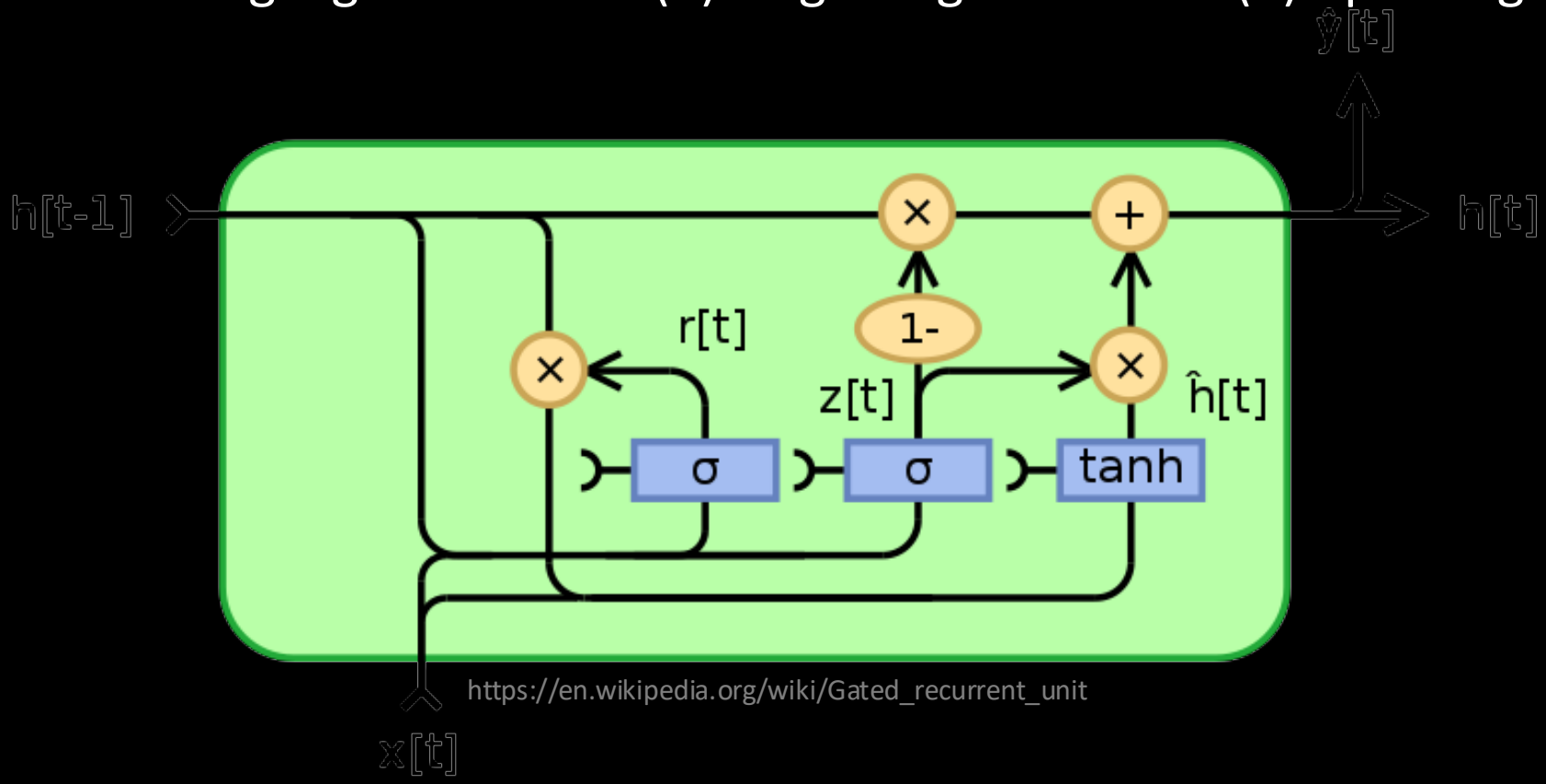
$$s_t = s_{t-1} \circ f + g \circ i$$

$$\frac{\partial s_t}{\partial s_{t-1}} = f$$

http://adventuresinmachinelearning.com/recurrent-neural-networks-lstm-tutorial-tensorflow/

Simplified LSTM: Gated Recurrent Unit (GRU)

Simplifies LSTM: single gate controls (1) forgetting factor and (2) updating hidden units



https://en.wikipedia.org/wiki/Gated_recurrent_unit

K. Chou et al; Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation; 2014

Today's Topics

- Limitation of feedforward neural networks: supporting sequential data
- Key insight: *recurrent* neural networks (RNNs)
- Breakthrough: gated RNNs for dynamically deciding what to remember
- Programming tutorial: recurrent neural networks in PyTorch

Today's Topics

- Limitation of feedforward neural networks: supporting sequential data
- Key insight: *recurrent* neural networks (RNNs)
- Breakthrough: gated RNNs for dynamically deciding what to remember
- Programming tutorial: recurrent neural networks in PyTorch

The End